POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Sébastien LE DIGABEL, président
Antoine LESAGE-LANDRY, membre et directeur de recherche
Antoine LEGRAIN, membre

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES xi

LIST OF SYMBOLS AND ACRONYMS xiii

CHAPTER 1 INTRODUCTION 1
- 1.1 Problem statement 2
- 1.2 Contributions 4
- 1.3 Canonicalization 4
  - 1.3.1 Conic reformulation 6
  - 1.3.2 `CVXPY` 7
- 1.4 Computing derivatives 10
- 1.5 Matrix computations 13
  - 1.5.1 Dense matrices 13
  - 1.5.2 Sparse matrices 14

CHAPTER 2 LITERATURE REVIEW 16
- 2.1 Matrix stuffing in `CVXPY` 16
- 2.2 Sparse derivatives by structure discovery 22
  - 2.2.1 `SparseConnectivityTracer` 24
  - 2.2.2 `CasADi` 25
- 2.3 Coefficient extraction from declared structure 25
  - 2.3.1 `AMPL` 26
  - 2.3.2 `JuMP` 26

CHAPTER 3 A lightweight canonicalization engine 27
- 3.1 Overview of the engine 27
- 3.2 Matrix representations 29

3.2.1 The permuted dense matrix 29
3.2.2 The stacked permuted dense matrix 32
3.3 Computational kernels 32
3.3.1 Atom implementations 32
3.3.2 Kernel dispatch 33
3.3.3 Reusing work across passes 34
3.4 Revisiting the limitations of matrix stuffing 35
3.5 The API and the CVXPY interface 36

CHAPTER 4 Benchmark results 38
4.1 Benchmark Setup 38
4.2 Single-solve compilation against `cvxcore` 39
4.3 Parametric compilation against `cvxcore` 39
4.4 Comparison against structure discovery: `CasADi` and `SCT` 41
4.5 Comparison against declared structure: `AMPL` and `JuMP` 44

CHAPTER 5 CONCLUSION 48
5.1 Summary of Works 48
5.2 Limitations 48
5.3 Future Research 48

REFERENCES 49

LIST OF TABLES

Table 1.1 A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output. 5

Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument f_1 with augmented matrix $[A_1\ b_1]$. 18

Table 2.2 Differentiation methods classified by the four choices of this section: the *expression type* (scalar or vectorized), the *mode* (forward, reverse, or both), the *assembly* of the Jacobian (formed *directly* by composing local Jacobians, or matrix-free through an *oracle* that returns only matrix–vector products and is reconstructed by seeded sweeps), and the *data format* (sparse or dense). The vectorized, forward, direct, sparse methods (lower block) need no graph coloring, unlike the matrix-free methods above them. 25

Table 2.3 Comparison of problem-data extraction methods across modeling languages. 26

Table 3.1 The atom classes and their shared Jacobian recipes. 33

Table 4.1 Cold compilation time across all 25 benchmarks, `diffengine` versus `cvxcore`. The ratio is `diffengine`/`CPP`; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios. 40

Table 4.2 Parametric compilation, `diffengine` versus `cvxcore`: first solve and warm re-solve. The ratio is `diffengine`/`CPP` on the re-solve; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios. 41

Table 4.3 Single problem-data extraction, `diffengine` versus the two structure-discovery systems: the `SCT` stack on its four ported problems and `CasADi` on all 25 benchmarks. Dashes in the `SCT` column mark problems not ported to the Julia harness. The ratio is `diffengine`/`CasADi`; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios. 43

Table 4.4 Parametric extraction, `diffengine` versus `CasADi`: first extraction and warm re-solve. The ratio is `diffengine`/`CasADi` on the re-solve; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios. 44

Table 4.5 Single problem-data extraction against the declared-structure systems, on the 21-problem ported subset. The `AMPL` pipeline is split into its stages: binary `.nl` write by the C translator, ASL read (parse and tape build), and derivative evaluations (constraints, Jacobian, Hessian of the Lagrangian, gradient). Stage columns come from one instrumented run while totals are medians of three, so stages need not sum exactly to the total; on the millisecond-scale problems the instrumented cold-cache stages exceed the warmer median total. Dashes: SlowPruning exceeds memory in both systems; `AMPL` rejects OptimalAdvertising (non-finite coefficient). 46

LIST OF FIGURES

Figure 1.1 Example of an affine expression tree for $Ax - b$. Variables are drawn as circles and constants (or parameters) as boxes. 6

Figure 1.2 *Before canonicalization.* `CVXPY` represents the DCP problem (1.5) as these expression trees. Variables are labeled with a circle, and parameters with a square. 9

Figure 1.3 *After canonicalization.* `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable x appears as a leaf of several trees; all occurrences refer to the same variable. 10

Figure 2.1 The vec convention on a matrix variable X and the product FX, with $X, F \in \mathbf{R}^{2\times 2}$. The entries of $\mathbf{vec}(X)$, in column-major order, index the columns of the Jacobians. 17

Figure 2.2 Matrix stuffing of problem (1.5) by a recursive walk of the conic-form expression trees of 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children. 19

Figure 2.3 What each layout stores during the walk, at the node $-(Fx - g)$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns. 20

Figure 2.4 The DPP path on the objective data of problem (1.5): the parameter map Q is built and cached once, and each re-solve recovers (c, d) with the single sparse product $Q\tilde{\theta}$. 20

Figure 2.5 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the parameter-free tree is re-walked on every change of the parameters. . 21

Figure 3.1 A permuted dense matrix. The dense block X is stored once, contiguously; the index vectors r and c place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored. 30

Figure 3.2 Row indexing of the permuted dense matrix A of Figure 3.1. One output row is zero, one block row is duplicated, and the column structure is unchanged: the result has row vector $(0, 2, 3)$, the column vector of A, and a block made of rows of X. 31

Figure 3.3 Double dispatch of the chain-rule products $B^T A$ and $B^T DA$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its color indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel. 33

Figure 3.4 The two passes of the `sum` atom. During init, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The value passes of the other structural atoms have the same form. 35

LIST OF SYMBOLS AND ACRONYMS

AD	Algorithmic Differentiation
AML	Algebraic Modeling Language
API	Application Programming Interface
ASA	Affine-Solver-Affine
BLAS	Basic Linear Algebra Subprograms
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

CHAPTER 1 INTRODUCTION

Since the early days of linear programming, practitioners of optimization have needed to translate mathematical formulations into data that solvers can process. At first, this meant encoding each coefficient of the problem matrices by hand, a process that can both be tedious and error-prone [1]. *Matrix generators* were introduced in the late 1960s to automate this task: they compute the coefficients through explicit loops and write them out in solver-readable formats like MPS [2, 3].

Matrix generators were able to remove most of the manual labor, but a gap still remained. Modelers naturally think in terms of sets, indices, and algebraic expressions, while procedural code constructs the coefficient matrices entry by entry. This mismatch between how a model is expressed and how it is implemented made them difficult to verify, modify, and communicate with others [4]. *Algebraic modeling languages* such as `AMPL` [5], `AIMMS` [6] and `GAMS` [7] closed this gap by accepting problem descriptions that closely follow the math. The survey by Orchard-Hays [8] gives a more complete overview of the early history of modeling languages.

More recently, modeling languages embedded in general-purpose programming languages have gained popularity. These include `Pyomo` [9,10] in Python, `JuMP` [11] in Julia, `CasADi` [12] in C++, and `YALMIP` [13] in MATLAB. Other domain-specific modeling languages specialized for convex optimization [14] are built on *disciplined convex programming* (DCP), the composition rules under which the convexity of a problem can be verified automatically [15, 16]. These DCP-based languages include `CVX` [17] in MATLAB, `CVXR` [18] in R, `CVXPY` [19] in Python, and `Convex.jl` [20] in Julia.

In the meantime, modern general purpose solvers [21–23] have mostly transitioned towards conic programming which generalizes linear programming and can represent nearly every convex program used in practice [24]. The translation task remains a responsibility of the modeling languages which now target conic problem data instead of LP coefficient matrices. In DCP-based languages, this translation process is called *canonicalization*. While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) it can also be significant. The main goal of this thesis is to reduce the compilation time of `CVXPY`.

1.1 Problem statement

The standard form of a convex optimization problem is

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & f_i(x;\theta) \leq 0, \quad i = 1,\ldots,m_1 \\ & g_j(x;\theta) = 0, \quad j = 1,\ldots,m_2 \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ is the optimization variable, the objective f_0 and inequality constraints $f_1,\ldots,f_{m_1}$ are convex and the equality constraints $g_1,\ldots,g_{m_2}$ are affine [14]. The parameter $\theta \in \mathbf{R}^p$ specifies data that can change between solves but is fixed and given when solving an instance of the problem. We call the parametrized problem (1.1) a problem *family*; for a fixed value of θ, we call it a problem *instance*.

Quadratic conic form. Canonicalization converts the high-level description (1.1) to a *quadratic conic form* used by many general-purpose convex solvers

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}x^T P(\theta)x + c(\theta)^T x \\ \text{subject to} & b(\theta) - A(\theta)x \in K \end{array} \tag{1.2}$$

where the *problem data* are the symmetric positive semidefinite matrix $P(\theta) \in \mathbf{S}^n_+$, the matrix $A(\theta) \in \mathbf{R}^{m\times n}$, the vectors $b(\theta) \in \mathbf{R}^m$ and $c(\theta) \in \mathbf{R}^n$ and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Problem (1.2) generalizes both quadratic programs (QPs), when $K = \mathbf{R}^m_+$ is the nonnegative orthant, and standard conic programs, when $P(\theta) = 0$.

Canonicalization in `CVXPY`. Most of the cost of canonicalization lies in the final step, where symbolic expression trees are converted to numerical problem data. This step is often called *matrix stuffing* or the *backend* of canonicalization. During the first solve of a problem family, `CVXPY` forms and caches a *parameter tensor* that maps the parameter vector to the problem data. On subsequent solves, new problem data can be obtained with a single matrix-vector product [25].

For large-scale problems, forming the parameter tensor can be prohibitively expensive, so `CVXPY` lets users opt out of this option entirely. Users must thus choose between paying a large upfront cost on the first solve and repeating the full canonicalization, with nothing reused, at every solve. We review `CVXPY` and the frontend of canonicalization in §1.3, as well as the parameter tensor and matrix stuffing in §2.1.

Matrix stuffing as a derivative. The constraint matrix A is the Jacobian of the stacked constraint expressions and P is the Hessian of the objective, so matrix stuffing amounts to computing a sparse Jacobian and Hessian. Classical methods build these matrices column by column with multiple passes of algorithmic differentiation (AD), using graph coloring to group columns that can be recovered in a single pass [26, 27]. These methods treat the expression as a black box: the sparsity pattern must first be detected and the columns grouped before a single entry of the matrix is obtained. We review the basics of computing derivatives in §1.4 and present sparse AD and graph coloring methods in §2.2.

Other approaches. A modeling language, by contrast, knows which variables enter each constraint. `AMPL` [5] is split in two components: its commercial *translator* writes the structure of the model to a `.nl` file [28], and the open source AMPL Solver Library (ASL) evaluates constraints and their derivatives from this file inside the solver process [29]. `Pyomo` [9] also interfaces to solvers through `.nl` files, among other formats, though it generally writes them more slowly than the `AMPL` translator. On the other hand, `JuMP` [11] has no single standard form to target. The model is built constraint by constraint, and each constraint stores its coefficients directly, so extraction reduces to copying them into the format of the chosen solver [30]. We review both these modeling languages in §2.3.

Our approach. We make use of the fact that the sparsity patterns of A and P are fixed across a problem family, since the parameter θ changes the values of the problem data but not their structure. We can therefore split matrix stuffing in a symbolic and numeric phase. The symbolic phase runs once on the first solve, where it computes the sparsity patterns, allocates memory for all intermediate matrices, and caches other computations that remain valid across the problem family. These patterns are known analytically from the atoms, so nothing needs to be detected and no graph coloring is needed. The numeric phase then fills in new values at every solve. This separation between structure and values is inspired by the interfaces of nonlinear programming solvers such as `IPOPT` [31].

The symbolic phase is much cheaper to form and cache than the parameter tensor, and the numeric phase is often similar in cost to applying the matrix-vector product. The problem data is also assembled entirely in memory rather than through a separate model file. Finally, we exploit structure within the sparsity patterns themselves, which often contain dense blocks of data. When possible, we store these blocks as dense matrices and compute with dense linear algebra kernels, rather than always relying on generic sparse formats and routines (we review dense and sparse matrix computations in §1.5).

1.2 Contributions

We make the following contributions:

1. We build `SparseDiffEngine`, a C library with Python bindings that computes sparse Jacobians and Hessians without graph coloring.
2. We implement a canonicalization backend for `CVXPY` built on `SparseDiffEngine`; this backend avoids the large upfront cost of the parameter tensor while still caching computations for subsequent solves.
3. We benchmark this backend against existing canonicalization and sparse derivative approaches.

All code contributions are open source and available at

`https://github.com/SparseDifferentiation/SparseDiffengine`
`https://github.com/cvxpy/cvxpy`
`https://github.com/SparseDifferentiation/Jacobian-accumulation`

Thesis outline. The rest of this chapter (§1.3 to §1.5) presents some important background on canonicalization, computing derivatives, and matrix computations. §2 reviews the three existing methods: `CVXPY`'s matrix stuffing (§2.1), sparse derivatives by detection and coloring (§2.2), and problem data from declared structure in `AMPL` and `JuMP` (§2.3). In §3 we present an overview of the main data structures and algorithms in `SparseDiffEngine` as well as its interface in `CVXPY`. Finally, §4 benchmarks this engine against the existing approaches, and concludes with a summary of our contributions and future improvements to the canonicalization backend in §5.

1.3 Canonicalization

The problem family (1.1) is written in *functional* form: an objective and constraint functions of the variable and the parameters. In a modeling language, the optimization variables $x_1, \ldots, x_k$ are collected into a single vector $x \in \mathbf{R}^n$, and likewise the parameters $\theta_1, \ldots, \theta_l$ into $\theta \in \mathbf{R}^p$ [14]. Users are not required to write constraints with a zero right-hand side as in (1.1): modeling languages accept inequalities with a convex left-hand side and a concave right-hand side, and equalities with both sides affine; moving every term to one side recovers the form (1.1). Internally, each function is represented as an expression tree.

Atoms. Modeling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in `CVXPY` since they are the fundamental building blocks of the modeling language [15]. Alongside its numerical definition, each atom declares useful attributes such as its *curvature*, *monotonicity* with respect to each argument, and the *sign* of its output. A selection of common atoms is shown in 1.1.

Table 1.1 A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output.

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`exp`	e^x	convex	non-decreasing	positive
`log`	$\log x$	concave	non-decreasing	any
`norm`	$\lVert x \rVert_2$	convex	unknown	nonnegative
`max`	$\max_i x_i$	convex	non-decreasing	any
`sum_largest`	sum of k largest entries	convex	non-decreasing	any

Expression trees. An expression is defined recursively as an atom evaluated at *subexpressions*, each of which is itself another expression. Such an expression is naturally stored as an *expression tree*, with atoms at its internal nodes and variables, parameters, and constants at its leaves. In Python, this tree can be formed automatically from user code through operator overloading. Most operations on an expression, from evaluation to convexity verification to differentiation, traverse the tree and combine the intermediate results at each atom. 1.1 shows the tree of the affine expression $Ax - b$.

Disciplined convex programming. When f_0 and all f_i are convex and all g_j are affine, (1.1) defines a convex optimization problem. To certify convexity, DCP-based modeling languages use the DCP composition rules to classify the curvature of each function [15]. Once all functions are classified, a problem is *DCP-compliant* if the objective and each constraint have curvatures compatible with a convex problem: a convex objective, convex $\leq$ concave inequalities, and affine equalities.

Otherwise, if a curvature is incompatible (*e.g.* f_0 is concave) or cannot be determined, the modeling language raises an error and terminates. Because these rules are only sufficient for

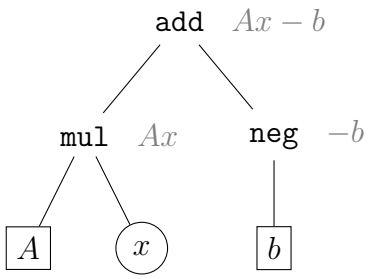


Figure 1.1 Example of an affine expression tree for $Ax - b$. Variables are drawn as circles and constants (or parameters) as boxes.

convexity, a problem that is genuinely convex but not expressed in a recognized form may still be rejected.

Composition rules. The composition $\phi(f_1(x), f_2(x), \ldots, f_k(x))$ of an atom ϕ with subexpressions f_i is convex when ϕ is convex and, for each argument $i \in \{1, \ldots, k\}$, one of the following holds:

- ϕ is non-decreasing in argument i and f_i is convex, or
- ϕ is non-increasing in argument i and f_i is concave, or
- f_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require ϕ and all of its arguments to be affine [15].

1.3.1 Conic reformulation

Functional conic form. Once a problem is verified to be DCP-compliant, canonicalization rewrites its functional form (1.1) into an equivalent *functional conic form*

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & h_i(x;\theta) \in \mathcal{K}_i, \quad i = 1, \ldots, m, \end{array} \tag{1.3}$$

where f_0 is affine or quadratic, each h_i is affine, and each $\mathcal{K}_i$ is one of the standard convex cones: the zero, nonnegative, second-order, power, exponential, and positive semidefinite cones.

Conic reformulation. Every nonlinear atom is rewritten into conic constraints in three steps. The first step rewrites the problem into *Smith form*, where each nonlinear atom is replaced by a new variable coupled with an equality constraint; every atom then acts only on affine arguments [32]. For a convex atom ϕ, the Smith form gives an auxiliary variable t with the equality constraint $\phi(x) = t$. The second step relaxes this equality to the inequality $\phi(x) \leq t$. The DCP rules allow a convex atom only where its value is minimized or upper-bounded, so the solver always prefers t as small as possible; the relaxation is therefore *lossless*.

The pair (x, t) then lies in the *epigraph* $\{(x, t) : \phi(x) \leq t\}$ of ϕ, so we often refer to t as an *epigraph variable*. A concave atom relaxes to $\phi(x) \geq t$ instead, giving a *hypograph variable*. The third and final step expresses each epigraph (hypograph) inequality as the membership of an affine function of (x, t) in a convex cone [33].

Example of reformulations. A first example is the ℓ_2 norm atom $\|x\|_2$, whose epigraph $\|x\|_2 \leq t$ is exactly the second-order cone constraint $(t, x) \in \mathcal{K}_{\text{soc}}$. A second example is the nondifferentiable absolute value atom $|x|$, whose epigraph $|x| \leq t$ can be written smoothly as the pair of affine inequalities $-t \leq x \leq t$. A final example is the sum of the k largest entries of x, which expands into a small linear program (LP): the epigraph sum_largest$(x, k) \leq t$ holds if and only if there exist a scalar q and a vector u such that

$$q\mathbf{1} + u \geq x, \qquad u \geq 0, \qquad kq + \mathbf{1}^\top u \leq t. \tag{1.4}$$

Here q acts as a threshold and each u_i measures how far the entry x_i rises above it, so that the sum of the k largest entries is bounded by $kq + \mathbf{1}^\top u$ [14].

1.3.2 `CVXPY`

`CVXPY` is a DCP-based modeling language embedded in Python [19], which began in 2013 as an implementation of the modeling principles of `CVX` [17]. Three fundamental design decisions differed from `CVX` at the beginning. The first is modularity: a few core data structures are shared by separate components for verification, canonicalization, matrix stuffing, and solver interfaces. The second is the expression tree as the central data structure, which is formed automatically from user code through Python's operator overloading. The third is support for parameters: since canonicalization never looks at the values of constants, much of the compilation work can be reused when a problem is solved repeatedly.

These ideas matured with version 1.0, which redesigned canonicalization as a general reduction-based rewriting system [34]. The new design enabled the addition of other disciplined gram-

mars [35–37], and the early parameter support grew into DPP, the restricted grammar under which canonicalized programs can be updated efficiently [25].

The open source project has since grown through contributions from many other enthusiastic developers; a recent paper surveys the new canonicalization and modeling features introduced between versions 1.1 and 1.9 [38]. The exposition of canonicalization in this chapter follows this survey, with more emphasis on the non-DPP path.

Reductions. The rewriting from the functional form (1.1) to the functional conic form (1.3) is carried out as a sequence of *reductions*. Each reduction provides an `apply` step that converts a problem into an equivalent one, and a `retrieve` step that recovers a solution of the original problem from a solution of the rewritten one [34].

A first example is `FlipObjective`, which turns a maximization problem into a minimization one by flipping the sign of the objective; its `retrieve` step simply flips the sign back.

A second example, `CvxAttr2Constr`, handles variable attributes, which are properties declared on a variable that restrict its domain. These include nonnegativity (`nonneg`), positive semidefiniteness (`PSD`), and explicit variable bounds, amongst others. When the target solver supports an attribute natively, `CVXPY` leaves it for the solver to enforce; otherwise the reduction creates a new variable that carries no attributes and adds explicit constraints that enforce the attribute. Its `retrieve` step then maps the reformulated solution back onto the original attributed variable.

The `Dcp2Cone` reduction. The main reduction, `Dcp2Cone`, applies the epigraph reformulation of §1.3.1 atom by atom: the problem is first rewritten into Smith form, each resulting equality is relaxed to an epigraph (or hypograph) inequality, and each inequality is finally expressed as a cone membership [32, 33]. In practice the three steps are performed at once: each nonlinear atom is replaced directly by its epigraph variable and cone constraints, and the intermediate Smith form and its relaxation never appear explicitly.

Example of canonicalization. To show a concrete example of the frontend of canonicalization, we consider a simple fitting with regularization problem

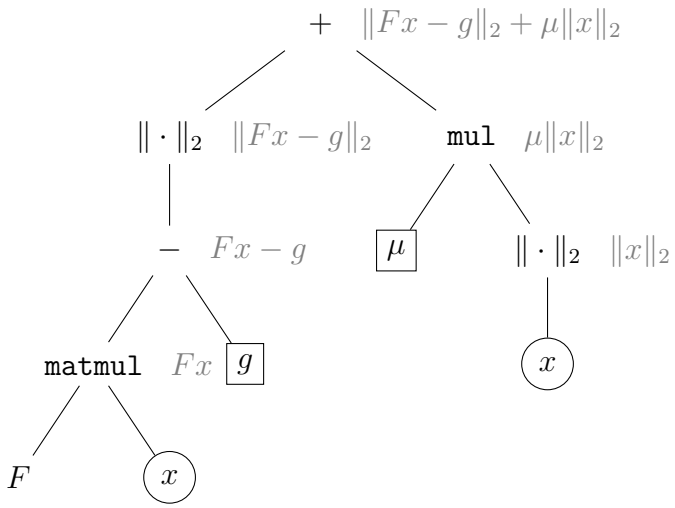
$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu\|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array} \tag{1.5}$$

with variable $x \in \mathbf{R}^n$, fixed problem data $F \in \mathbf{R}^{m\times n}$, and parameters $g \in \mathbf{R}^m$ and $\mu \geq 0$; the constraint $x \succeq 0$ requires every entry of x to be nonnegative. The problem formulation

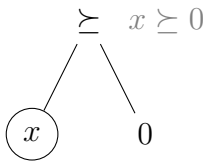
in CVXPY code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library NumPy [39] for the data arrays. The expression tree representation of the problem is shown in 1.2.



(a) Expression tree of the objective



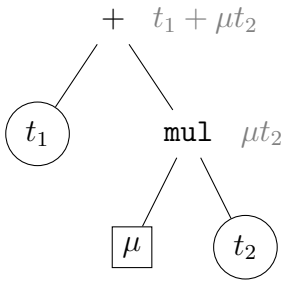
(b) Expression tree of the constraints

Figure 1.2 *Before canonicalization.* CVXPY represents the DCP problem (1.5) as these expression trees. Variables are labeled with a circle, and parameters with a square.

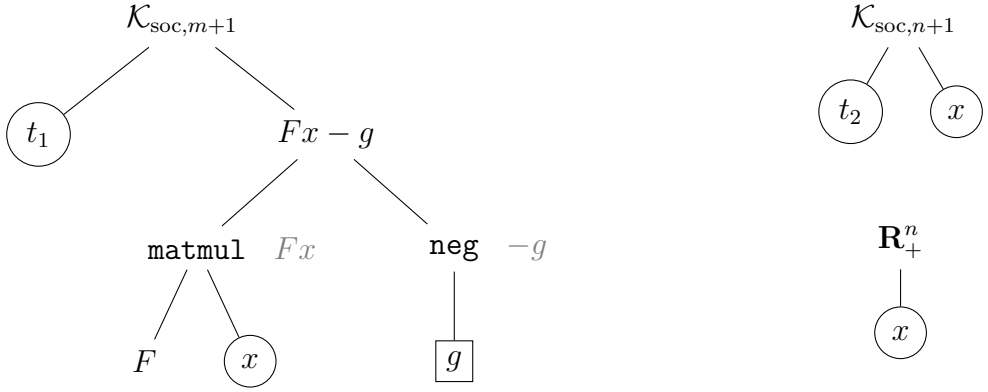
Conic form expression tree. After applying the reductions, the problem has been *canonicalized* to

$$\begin{array}{ll} \text{minimize} & t_1+\mu t_2 \\ \text{subject to} & x\in\mathbf{R}^n_+ \\ & (t_1,Fx-g)\in\mathcal{K}_{\text{soc},m+1} \\ & (t_2,x)\in\mathcal{K}_{\text{soc},n+1} \end{array} \tag{1.6}$$

where t_1 and t_2 are new epigraph variables, and the cone $\mathcal{K}$ from 1.2 is the product $\mathbf{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. Problem (1.6) is now in the functional conic form of §1.3.1: every constraint is a cone membership, and the objective and constraint expressions are affine. Comparing the DCP trees of 1.2 with the conic-form trees of 1.3 shows the effect of canonicalization: the nonlinear norm atoms have been replaced by epigraph variables, leaving affine expression trees in the variables. The three rows of (1.6) remain three separate cone constraints, each carrying the affine expressions that must lie in its cone; 1.3 accordingly shows one expression tree per constraint, rather than a single combined tree.



(a) Expression tree of the objective



(b) One expression tree per constraint

Figure 1.3 *After canonicalization.* `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable x appears as a leaf of several trees; all occurrences refer to the same variable.

1.4 Computing derivatives

There are many methods for computing the derivatives of a function, each with its own trade-offs [40]. This section surveys the elementary ones; §2.2 develops the machinery for

sparse derivatives and positions our approach among the systems that implement it.

The Jacobian. For a vector-valued map $F : \mathbf{R}^n \to \mathbf{R}^m$ of an input $x \in \mathbf{R}^n$, the first-order derivative is the *Jacobian* $J \in \mathbf{R}^{m \times n}$ of partial derivatives of the outputs with respect to the inputs,

$$J_{ij} = \frac{\partial F_i}{\partial x_j}, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n. \tag{1.7}$$

In this thesis, the map of interest stacks the affine constraint expressions of a canonicalized problem into a single vector function of the variables; its Jacobian is the constraint matrix A of (1.2).

The Hessian. For a scalar function $f : \mathbf{R}^n \to \mathbf{R}$, the second-order derivative is the *Hessian* $H \in \mathbf{R}^{n \times n}$ of second partial derivatives,

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}, \qquad i, j = 1, \ldots, n. \tag{1.8}$$

The Hessian is the Jacobian of the gradient of f, and is symmetric whenever f is twice continuously differentiable. It is therefore obtained by differentiating the gradient, with no method beyond those used for the Jacobian. Here the scalar function of interest is the canonicalized objective, and its Hessian is the matrix P of (1.2).

Numerical differentiation. Other than computing derivatives by hand, the simplest method is to use *numerical differentiation*, which approximates the derivative of a function at a point by evaluating the function at nearby points using a *finite difference* formula such as

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{1.9}$$

where h is a small step size (the central variant $f'(x) \approx (f(x+h) - f(x-h))/2h$ improves accuracy at the cost of one more evaluation). The approach is simple but has two downsides. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to *truncation errors* (when h is too large) or *rounding errors* (when h is too small) in the computed derivative if the step size is not carefully chosen [41]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations (also known as a *derivative checker*) [42].

Symbolic differentiation. Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This *symbolic* approach to differentiation is used in computer algebra systems such as `Mathematica` [43] and `Maple` [44] as well as the `SymPy` library in Python [45]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can quickly form exponentially large symbolic expressions, which is impractical and inefficient for many problems [46]. Interestingly, when intermediate expressions are stored as *common subexpressions*, and the expression is represented with a *directed acyclic graph* (DAG) instead of a tree, symbolic differentiation performs the same set of operations as *reverse mode differentiation* [47].

Algorithmic differentiation. *Algorithmic differentiation* treats the function as a *computational graph* of elementary operations, each with a known local derivative rule. The chain rule is then applied to combine these local derivatives throughout the graph, giving the derivative of the whole function. AD is also commonly referred to as *automatic differentiation*; we prefer the term *algorithmic* to emphasize that AD differentiates the algorithm that computes a function, rather than a numerical approximation or a symbolic formula [46]. In this broad sense, AD can refer to any method that applies the chain rule over a computational graph; §2.2 surveys the systems that implement it and the choices that distinguish them.

Modes of differentiation. The chain rule can be applied in different orders, all giving the same result. The two best-known are *forward mode* (or *tangent* mode), which propagates derivatives from the inputs toward the outputs, and *reverse mode* (or *adjoint* mode), which propagates them from the outputs back toward the inputs [40]. The two modes have asymmetric costs: in forward mode the work scales with the number of inputs, and in reverse mode with the number of outputs. Reverse mode is therefore preferred when there are far fewer outputs than inputs, such as computing the gradient of a scalar function. Other mixed-ordering modes exist, commonly grouped under the umbrella term *cross-country* elimination [46]. However, finding the best ordering on a general graph is difficult, so these modes are rarely used in practice [48].

1.5 Matrix computations

Matrix computations appear at every stage of this thesis. The problem data P and A are sparse matrices, and matrix stuffing assembles them from smaller intermediate matrices. This section reviews the two families of matrix representations, dense and sparse, each from the standpoint of both storage and algorithms.

1.5.1 Dense matrices

Dense storage. A dense matrix is stored as a single contiguous array of its entries, ordered row by row (row-major) or column by column (column-major). The position of an entry is computed directly from its indices, so no index arrays are needed, and rows or columns are traversed sequentially in memory.

The BLAS. Dense linear algebra is served by a standardized interface of vector and matrix kernels, the *Basic Linear Algebra Subprograms* (BLAS). The interface is organized in three levels by the ratio of computation to data: level 1 covers vector operations such as dot products and scaled additions [57], level 2 covers matrix–vector products, and level 3 covers matrix–matrix products [58]. LAPACK, the library behind dense factorizations and linear system solving in most scientific software, is built on top of these kernels [59]. Because the interface is standard, decades of optimization effort concentrate behind it: implementations such as `OpenBLAS`, Intel `MKL`, and Apple `Accelerate` are tuned to each processor family, and every program calling the interface benefits. In practice, an optimized implementation is available on nearly every machine, whether shipped with the operating system or bundled by numerical libraries.

Why dense kernels are fast. The performance of optimized BLAS implementations comes from working with, rather than against, the memory system. A modern processor multiplies numbers far faster than it loads them from main memory, so fast code must reuse data already sitting in its caches and must read memory contiguously. Dense storage makes both possible: optimized kernels traverse arrays in memory order, process several entries per instruction with vector (SIMD) units, and tile matrix–matrix products into blocks that fit in cache [60]. Level-3 kernels benefit the most: they perform $O(n^3)$ operations on $O(n^2)$ data, so each entry loaded from memory is reused many times. These speeds, however, rely on every entry being stored and processed, zeros included.

1.5.2 Sparse matrices

In optimization problems, the matrices P and A contain mostly zero entries; such matrices are called *sparse*. Storing only the nonzero entries makes memory and work scale with their number rather than with the product of the dimensions. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$S = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix} \tag{1.10}$$

can be represented in coordinate format as

```
int    rows[] = {0, 1, 1,  1,  2,  2};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we use zero-based indexing and array declaration notation from the C programming language [49]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

Compressed sparse row format. The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (1.10) can be represented in CSR format as

```
int  idxptr[] = {0, 1,         4,      6};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [50].

Compressed sparse column format. The *compressed sparse column* or CSC format applies the same compression to the columns instead: `idxptr` now points into a `rows` array, and the nonzero entries are stored column by column. The CSC arrays of a matrix read exactly as the CSR arrays of its transpose, so transposition is a conversion between the two formats, computable without sorting in one counting pass over the nonzeros [52]. Accessing a column of a CSR matrix instead requires a scan of every row, so algorithms that traverse columns prefer CSC; direct methods for sparse linear systems are the standard example [51].

Choice of format. The access pattern of the algorithm decides: a computation that traverses rows wants CSR, one that traverses columns wants CSC. Since conversion costs a single counting pass, keeping the same matrix in both formats is common. The engine developed in this thesis (§3) does exactly this, storing Jacobians in CSR and pairing the two formats in its matrix products.

Matrix-vector products. Both compressed formats multiply a dense vector in one pass over the nonzeros, but the loop must follow the storage: CSR *gathers*, computing each output entry as a sparse dot product along its row, while CSC *scatters*, accumulating each column's contribution into the output vector.

Matrix-matrix products. The product of two sparse matrices is harder than the operations above because the sparsity pattern of the output is not known in advance. Implementations therefore split the work in two phases: a *symbolic* phase determines the pattern and allocates the output, and a *numeric* phase fills its values [52, 53]. The split rewards reuse, and the engine of §3 is built around it: the symbolic phase runs once per problem family and only the numeric phase repeats.

Other formats. The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [54] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

The standard references develop these data structures for solving sparse linear systems, a task this thesis never performs, but they apply unchanged [51, 55, 56]; Gustavson's paper [52] is the origin of the two-phase product and the sort-free transposition used by the engine.

CHAPTER 2 LITERATURE REVIEW

Structure of the chapter This chapter reviews the systems this thesis builds on and compares against, each building on one background section of the introduction. §2.1 covers matrix stuffing, the backend of `CVXPY`'s canonicalization (§1.3.2), which assembles the problem data from the functional conic form. §2.2 reviews sparse algorithmic differentiation systems, which discover the structure of the derivative, and positions the approach of this thesis among them. §2.3 reviews modeling systems that extract the problem data from the declared structure of the model instead, as `AMPL` and `JuMP` do.

2.1 Matrix stuffing in `CVXPY`

`CVXPY` takes the perspective of matrix calculus [61] for computing the coefficient matrices of linear expressions; this is where most of the computational burden of canonicalization lies. Matrix stuffing is named for what it does: the coefficients of every canonicalized expression are stuffed into their designated slots in the sparse standard-form data handed to the solver. The name suggests bookkeeping, but the coefficients it assembles are the Jacobians and Hessians of the canonicalized expressions: matrix stuffing is a sparse derivative computation, a view developed in §1.4 and §2.2.

The section gives a brief summary of matrix calculus and a few examples, presents the conventions of `CVXPY`'s backend, and closes with implementation details that become important when contrasting it with the engine of §3.

Matrix calculus. Matrix calculus extends differentiation to functions whose arguments and values are vectors and matrices. Systematic rules for such derivatives go back to Dwyer and Macphail [62], and there are many ways to define the derivative of a matrix-valued expression with respect to a matrix variable [61]. The simplest is to vectorize: every expression is identified with

$$\mathbf{vec}(X) = \begin{bmatrix} x_{11} & \cdots & x_{m1} & x_{12} & \cdots & x_{mn} \end{bmatrix}^T, \tag{2.1}$$

the vector that stacks the columns of $X \in \mathbf{R}^{m \times n}$, and the derivative of one vectorized expression with respect to another is an ordinary Jacobian matrix. Under this convention matrix operations become linear operators on the vectorized arguments, and their Jacobians

carry structure: for example

$$\mathbf{vec}(FX) = (I \otimes F)\ \mathbf{vec}(X), \tag{2.2}$$

so the matrix product FX corresponds to the Kronecker-structured, block-diagonal Jacobian $I \otimes F$. The convention is illustrated in 2.1: as an expression, a matrix variable is the identity map of its own entries, so its Jacobian is an identity block whose columns are indexed by the entries of $\mathbf{vec}(X)$, and the Jacobian of the product FX is block-diagonal with one copy of F per column of X.

```
X = cp.Variable((2, 2))
```
$$\Longrightarrow \quad \frac{\partial\, \mathbf{vec}(X)}{\partial\, \mathbf{vec}(X)} = \begin{array}{c} \\ x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

(a) A matrix variable is the identity map of its own entries.

```
Y = F @ X
```
$$\Longrightarrow \quad \frac{\partial\, \mathbf{vec}(Y)}{\partial\, \mathbf{vec}(X)} = I \otimes F = \begin{array}{c} \\ y_{11} \\ y_{21} \\ y_{12} \\ y_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} f_{11} & f_{12} & 0 & 0 \\ f_{21} & f_{22} & 0 & 0 \\ 0 & 0 & f_{11} & f_{12} \\ 0 & 0 & f_{21} & f_{22} \end{bmatrix} \end{array}$$

(b) The Jacobian (2.2) of the product $Y = FX$ is block-diagonal, one copy of F per column of X.

Figure 2.1 The vec convention on a matrix variable X and the product FX, with $X, F \in \mathbf{R}^{2\times 2}$. The entries of $\mathbf{vec}(X)$, in column-major order, index the columns of the Jacobians.

Composing linear operators. After canonicalization every expression is affine, so each node of a conic-form tree can be written in the vectorized form

$$f(x) = A\,x + b = \begin{bmatrix} A & b \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix}, \tag{2.3}$$

where x is the stacked vector of all variables, A is the Jacobian of the node, constant since f is affine, and b is a constant offset. Matrix stuffing computes the augmented matrix $[A\ b]$ of

every node in one bottom-up pass. A variable leaf contributes an identity block in its own column slot, a constant leaf contributes its value to the offset column, and an interior atom g with arguments $f_1, \ldots, f_k$ is a linear map $\sum_i L_i f_i$ whose constant Jacobians L_i apply directly to the augmented matrices of its children,

$$\begin{bmatrix} A & b \end{bmatrix} = \sum_{i=1}^{k} L_i \begin{bmatrix} A_i & b_i \end{bmatrix}. \tag{2.4}$$

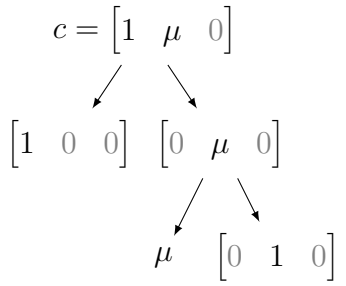
Computing the problem data thus amounts to composing many linear operators over the tree: the node matrices of the figures in this section are exactly these augmented blocks, and at the root of the constraint tree the augmented matrix is the problem data $[A\ b]$ itself.

Views and copies. The simple operators act as *views*, modifying the augmented matrix of their argument in place: negation flips the sign of every stored entry, indexing keeps the selected rows, and reshaping costs nothing at all, since it leaves the vectorization of the expression unchanged. The operators that create *copies* are mostly the bivariate atoms that carry data: one operand of a matrix product, an elementwise multiplication or a division is constant under the DCP rules, and the atom forms a matrix from that operand, multiplies it with the augmented matrix of the other, and stores the product as new data. An expository subset is classified in 2.1.

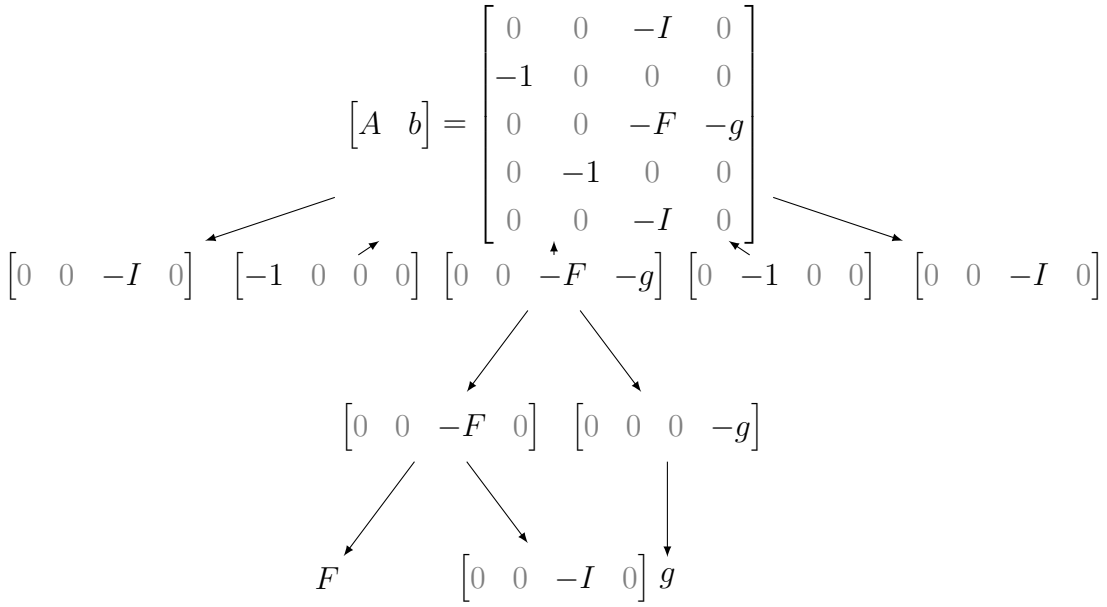
Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument f_1 with augmented matrix $[A_1\ b_1]$.

Atom	**Operator** L	**Action on** $[A_1\ b_1]$
Views		
$-f_1$	$-I$	flip the sign of every stored entry
f_1^T	P, the vec-permutation	permute the rows
$f_1[\mathcal{I}]$	$P_{\mathcal{I}}$, a row selection	keep the rows indexed by $\mathcal{I}$
reshape(f_1)	I	none, the vectorization is unchanged
Copies		
Ff_1	$I \otimes F$	multiply by a block diagonal formed from F
$f_1 B$	$B^T \otimes I$	multiply by a Kronecker matrix formed from B
$c \odot f_1$	$\mathbf{diag}(\mathbf{vec}(c))$	multiply by a diagonal formed from c
f_1/c	$\mathbf{diag}(\mathbf{vec}(c))^{-1}$	multiply by the reciprocal diagonal

Example. The walk on the conic-form trees of problem (1.5) is traced in 2.2: the leaves contribute their blocks and offsets, the interior nodes apply their operators, and the root stacks its children at their constraint row offsets into $[A\ b]$. The difficulty that remains is doing this efficiently for arbitrary problems.



(a) Matrix stuffing of the objective



(b) Matrix stuffing tree of the constraints

Figure 2.2 Matrix stuffing of problem (1.5) by a recursive walk of the conic-form expression trees of 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children.

Local versus global derivatives. The augmented matrices of (2.3) can be laid out in two ways (2.3). `CVXPY` differentiates *locally*, one variable at a time: each node stores one coefficient block per variable appearing beneath it, keyed by the variable's identifier, and every operator is applied to each block in turn. The constant data is stored under the key -1, a Python idiom for the last column, the offset b. The columns of a block index only

the entries of its own variable; only after the walk completes are the blocks shifted to their variables' column offsets and concatenated into the full matrix $[A\ b]$.

The alternative is a *global* derivative: a single Jacobian per node whose columns span the whole stacked vector x from the outset. The local layout keeps the blocks small and skips variables absent from a subtree, but every operator runs once per stored block; the engine of §3 takes the global route.

$$\left\{ \begin{array}{cc} x & 1 \\ x \mapsto \begin{bmatrix} -F \end{bmatrix}, & -1 \mapsto \begin{bmatrix} -g \end{bmatrix} \end{array} \right\} \qquad \begin{array}{cccc} t_1 & t_2 & x & 1 \\ \begin{bmatrix} 0 & 0 & -F & -g \end{bmatrix} \end{array}$$

(a) Local layout. (b) Global layout.

Figure 2.3 What each layout stores during the walk, at the node $-(Fx-g)$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns.

Parameters and tensors. DPP restricts parameters to enter expressions affinely, so the problem data is affine in the parameters [25], and `CVXPY` accounts for this with a third tensor axis: every coefficient block stores one slice per parameter entry plus one slice for the constant part, and the operators of the walk apply to each slice. Flattening the slices of the root yields the cached *parameter map*

$$\begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta}, \qquad \tilde{\theta} = \begin{bmatrix} \theta \\ 1 \end{bmatrix}, \tag{2.5}$$

shown in 2.4 for the objective of problem (1.5); the constraint data $[A\ b]$ is recovered the same way from its own map. After a parameter change, a re-solve costs a single sparse matrix–vector product $Q\tilde{\theta}$.

$$Q = \begin{array}{c} \\ t_1 \\ t_2 \\ x \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^T & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 1 \\ \mu \\ 0 \\ 0 \end{bmatrix}$$

Figure 2.4 The DPP path on the objective data of problem (1.5): the parameter map Q is built and cached once, and each re-solve recovers (c, d) with the single sparse product $Q\tilde{\theta}$.

The `ignore_dpp` path. Problems that fall outside the DPP grammar, or that opt out of it by solving with `ignore_dpp=True`, take a simpler path. The reduction `EvalParams` replaces each parameter leaf by a constant holding its current value, and the parameter-free tree that results is stuffed as if the problem had no parameters (2.5). The path is fully general, since arbitrary functions of parameters reduce to constants, but every change of θ repeats the whole symbolic walk even though the sparsity pattern of the problem data never changes. This is the setting the engine of §3 improves.

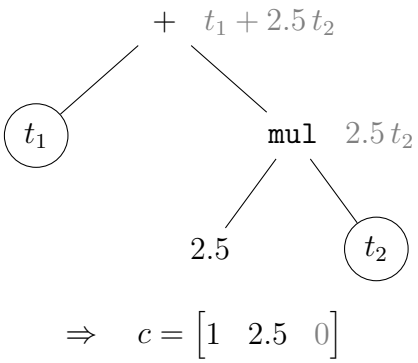


Figure 2.5 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the parameter-free tree is re-walked on every change of the parameters.

Code generation. A third option is to compile the problem ahead of time. `CVXPYgen` emits problem-specific C code that embeds the cached parameter map (2.5) of a DPP problem [63], continuing an earlier line of work on code generation for embedded second-order cone programs [64]. The benchmarks of §4 do not compare against it: code generation is an offline step that fixes the problem structure at compile time, a different deployment model from a runtime backend that must accept arbitrary problems.

Implementation details. Three implementation details, shared by `CVXPY`'s two backends, the C++ `cvxcore` and the `SciPy` backend, stand out as limitations of the current approach; the engine of §3 does each differently.

- *External sparse libraries.* Both backends delegate every matrix operation to a general-purpose sparse library, Eigen [65] for `cvxcore` and `scipy.sparse` [54] for the `SciPy` backend. The storage formats and kernels are fixed by the library, leaving no room to exploit structure known at canonicalization time.

- *Everything is sparse.* All coefficient data is treated as sparse: dense atom data is

converted to a sparse matrix on entry, and no operator carries a dense or block special case, so dense blocks pay sparse-format overhead and no BLAS kernel (§1.5.1) is ever used.

- *Quadratic objectives are separate.* Each quadratic form is swapped for a dummy variable, and the surrounding affine expression is stuffed by the same walk. The matrices of the quadratic forms are then scaled by the resulting coefficients and reassembled into the global objective matrix P.

References. Michael Grant's papers and PhD thesis are the standard references on verifying convexity and canonicalization [15, 16, 33]; the code-generation paper of Chu et al. [64] contributes the relaxed Smith form, and the Mosek modeling cookbook [66] complements them on epigraph reformulations. The original `CVXPY` paper [19] introduces the matrix calculus perspective, and the rewriting-system paper [34] situates matrix stuffing as the final step of the reduction chain. The DPP paper [25] develops the parameter map and its caching; the code generation lineage runs from the embedded second-order cone solvers of Chu et al. [64] to `CVXPYgen` [63].

2.2 Sparse derivatives by structure discovery

The introduction (§1.4) presented the elementary methods for computing derivatives: finite differences, symbolic differentiation, and AD with its forward and reverse modes. The software systems implementing AD differ in four basic choices: whether the function is treated as a *scalar* or *vectorized* expression, whether derivatives are propagated *forward* or in *reverse*, whether the Jacobian or Hessian is assembled *directly* or accessed through an *oracle* of matrix–vector products, and whether the resulting matrix is *sparse* or *dense*.

The problem data for an optimization model is often large and sparse, so we place particular emphasis on computing sparse derivatives in this section. The standard approach in the literature is scalar and oracle-based differentiation [46], where graph coloring is used to recover a sparse matrix in far fewer oracle evaluations. This thesis instead proposes to compute these sparse derivatives directly on a vectorized graph whose sparsity pattern follows from the structure of the operations, so no coloring is needed.

The remainder of the section presents the standard scalar and oracle-based pipeline, then the vectorized and direct alternatives, and closes by gathering a range of relevant works, organizing them in a table according to their particular approach to differentiation.

Scalar differentiation. The earliest and most foundational form of AD operates on a *scalar* computational graph, where every node is a single elementary operation (an addition, a multiplication, or a univariate function such as sin or exp), and every edge carries the corresponding scalar partial derivative [46, 67]. Since each elementary operation has a simple and known derivative, the chain rule reduces to multiplying and summing scalars along the graph. Early AD tools such as ADOL-C [68] and CppAD [69] form the scalar graph by recording a runtime trace of every elementary operation. Such a trace can be enormous: a single matrix–vector product Ax unrolls into one scalar multiplication for every entry of A, together with the additions that sum them, so it quickly reaches millions of nodes for a realistic problem.

Oracle-based differentiation. Instead of the Jacobian J itself, AD returns its action on a vector: the Jacobian-vector product (JVP) Jv in forward mode, and the vector-Jacobian product (VJP) $J^\top w$ in reverse mode (for the Hessian, the analogous Hessian-vector product Hv comes from one forward pass over one reverse pass) [40]. These matrix-vector products are obtained without ever forming J, each at about the cost of one function evaluation. Borrowing terminology from numerical linear algebra, we call this the *matrix-free*, or *oracle*, view of AD. The oracle is sufficient for many applications; they never form the matrix and use only its action on vectors, as in backpropagation for training neural networks or Krylov methods for linear systems.

Recovering the Jacobian. Recovering J explicitly requires *probing* the oracle with a sequence of seed vectors: the n unit seeds recover it column by column in forward mode, and the m unit seeds recover it row by row in reverse mode (the Hessian is recovered the same way, though its symmetry permits fewer seeds) [46]. When the Jacobian has many rows or columns this becomes prohibitively expensive, and reducing the number of probes is exactly the purpose of the graph-coloring methods described next. Crucially, coloring operates on the oracle alone, independently of how the underlying graph is built, so even fully vectorized frameworks rely on it when they need an assembled sparse Jacobian.

Graph coloring. The cost of probing drops when several seeds share a single pass. Two columns of J with no nonzero in a common row are *structurally orthogonal*, so one seed recovers both without their nonzeros colliding. Finding the fewest such groups is a graph-coloring problem on the column intersection graph, whose edges join columns that share a row [26, 27, 70]. The Hessian compresses further: its symmetry admits dedicated star and acyclic colorings that use fewer colors than the general Jacobian case [71].

Sparse derivatives. An oracle-based sparse Jacobian is obtained in three stages: *detect* the sparsity pattern, *compress* it by coloring, and *propagate* the compressed seeds through the oracle to recover the nonzero entries [72]. The pattern is computed once and reused, so its cost is amortized across repeated evaluations of the same function. This pipeline underlies sparse AD in oracle-based tools such as `CasADi` [12], which we examine at the end of this section.

Vectorized differentiation. More recent frameworks coarsen the graph so that each node is a whole vector or matrix operation rather than a single scalar. The local derivative of such a node is then a matrix, a block of the Jacobian, and these blocks are combined by the rules of matrix calculus [61]. The resulting graph has far fewer nodes than the scalar trace, which lets frameworks such as JAX [73] differentiate large programs efficiently. Vectorization alone, however, does not change the assembly: these frameworks still expose only the oracle, and still rely on coloring when they need an assembled sparse Jacobian.

Direct differentiation. Direct methods go a step further and abandon the oracle altogether. Rather than returning products, they propagate the local Jacobian blocks themselves through the graph and combine them to form J directly, so neither probing nor coloring is needed. Vectorization is the enabler: it gives each block algebraic structure, whereas a scalar node carries only a 0/1 dependency. Recent works apply this idea in statistics [74] and optimal control [75]. Direct propagation wins when the local blocks are well structured and intermediate fill-in stays bounded, while coloring through the oracle can still be cheaper when the intermediates densify. Which assembly is preferable ultimately follows the solver that consumes the result: the conic solvers targeted in this thesis require the matrix explicitly as data, which is exactly what the direct approach produces.

Overview of recent works. We now examine in more detail the two systems most relevant to this thesis: the sparse-AD stack of the Julia ecosystem, anchored by `SparseConnectivityTracer`, and `CasADi`, the system benchmarked against in §4.

2.2.1 `SparseConnectivityTracer`

`SparseConnectivityTracer` (SCT) is the sparsity-detection tool of the Julia sparse-AD stack developed by Hill and Dalle [72, 78]. It implements the *detect* stage of the detect–compress–propagate pipeline described above, using operator overloading to trace which inputs each output depends on.

Table 2.2 Differentiation methods classified by the four choices of this section: the *expression type* (scalar or vectorized), the *mode* (forward, reverse, or both), the *assembly* of the Jacobian (formed *directly* by composing local Jacobians, or matrix-free through an *oracle* that returns only matrix–vector products and is reconstructed by seeded sweeps), and the *data format* (sparse or dense). The vectorized, forward, direct, sparse methods (lower block) need no graph coloring, unlike the matrix-free methods above them.

Method / work	**Expression type**	**Mode**	**Assembly**	**Data format**
ADOL-C, CppAD [68, 69]	scalar	both	oracle	dense
`CasADi` [12]	scalar / vectorized	both	oracle	sparse
JAX [73]	vectorized	both	oracle	dense
`torch-sla` [76]	vectorized	reverse	oracle	sparse
`SCT` stack (Hill–Dalle) [72]	scalar	both	oracle	sparse
Zou and Jiang [75]	vectorized	forward	direct	sparse
Kwok et al. [74]	vectorized	forward	direct	sparse
∇SD [77]	vectorized	forward	direct	sparse
This thesis	**vectorized**	**forward**	**direct**	**sparse**

2.2.2 `CasADi`

`CasADi` [12] is an integrated symbolic-AD framework for optimization whose QP interface extracts the explicit matrices H and A by AD on the expression graph; it is the system this thesis benchmarks against in §4.

References. The book by Griewank and Walther [46] is a comprehensive reference on the theory and practice of scalar automatic differentiation, including the graph-elimination view of Jacobian accumulation. The book by Naumann [67] is another great reference that focuses on the implementation of AD in software, with a particular emphasis on the use of operator overloading and source code transformation techniques. The survey by Baydin et al. [40] is an excellent and relatively up-to-date overview of the various methods for automatic differentiation in machine learning. For the combinatorial side of sparse derivative computation, the survey of Gebremedhin et al. [27] is the standard entry point, and the illustrated guide of Hill and Dalle [79] gives an accessible modern tutorial on the detection–coloring–compression pipeline. Finally, the `CasADi` paper [12] is a self-contained reference covering its expression graphs, AD implementation, and solver interfaces.

2.3 Coefficient extraction from declared structure

A useful question sorts every modeling system: are the coefficients of the problem data *stored* as the user typed them, *differentiated* out of an expression graph, or *detected* at runtime and

recovered through coloring? The previous section covered the third answer; this section examines two mature systems that avoid discovery altogether, `AMPL` and `JuMP`. Both declare the structure of the problem up front, so extraction reduces to reading off what is already known.

2.3.1 `AMPL`

`AMPL` [5] is accessed from Python through the `amplpy` API [80]: the model is written to a compact binary `.nl` file [28] whose expression graphs the AMPL Solver Library (ASL) rebuilds inside the solver process [29].

2.3.2 `JuMP`

`JuMP` [11,81] is an algebraic modeling language embedded in Julia and built on the `MathOptInterface` (MOI) abstraction layer [30]. For the linear, quadratic, and conic problems considered here, coefficients are stored as the user types them, and no differentiation occurs anywhere on the problem-data path.

Table 2.3 Comparison of problem-data extraction methods across modeling languages.

	`JuMP`	`AMPL`	`CasADi`	`CVXPY`
Front-end paradigm	Algebraic modeling	Algebraic, file-based	Symbolic graphs + AD	Disciplined convex (DCP)
Problem classes	LP, MILP, QP, conic, NLP	LP, MILP, NLP (conic via recent drivers)	NLP, QP / conic (QP-shaped)	Convex: LP, QP, SOCP, SDP, EXP
Path to conic solvers	Native cones + bridges (MOI)	Solver-side, recent drivers	`soc` helper on the QP path	Automatic DCP reduction
Problem-data mechanism	Eager, per scalar constraint	`.nl` file; nonlinear via solver AD	Explicit H, A by AD on the graph	Affine-tree stuffing + parameter map

References. The MathOptInterface paper [30] is the best reference on the design of `JuMP`'s solver abstraction layer and its bridge system. David Gay's technical reports [28, 29] remain the authoritative description of the `.nl` format and the `AMPL` solver interface.

CHAPTER 3 A lightweight canonicalization engine

Fundamentally, there is not much difference between how `SparseDiffEngine` and `CVXPY`'s canonicalization backend compute derivatives. However, there are many interesting implementation details which we will present in this chapter. Firstly, the use of a custom sparse matrix data structure allows a natural split between the symbolic and numeric phases of the derivative computation. This separation is particularly useful for compiling problems involving parameters, where the structure is fixed but the numerical values can change. Secondly, in addition to being sparse, the data of most optimization problems also have a lot of structure, such as dense blocks or diagonality. In `SparseDiffEngine` we introduce two new matrix classes: `permuted-dense` and `stacked-permuted-dense` for this purpose; they enable us to make use of dense storage and BLAS subroutines when possible.

Finally, some important design decisions differ from `CVXPY`'s backends for simplicity rather than speed. Every node carries a single Jacobian whose columns span all of the variables of the problem, rather than a dictionary of local Jacobians keyed by variable. As a consequence, the chain rule is a single matrix product, and the problem data assembles by concatenation. In addition, quadratic functions pass through dedicated Hessian oracles instead of a separate substitution and matrix stuffing pass.

The rest of the chapter is structured as follows: we begin with a brief overview of the engine, describing how expressions are represented as trees of atoms and how the code is organized. We then present the matrix representations used for the derivative computations, where we will focus much of our attention on the `permuted-dense` matrix, which appears most often. Next we describe the algorithms, or *kernels*, which allow us to operate on and combine these matrix representations. Rather than attempting to explain too much, we focus on the organization of the code and on how we handle operations between different types of matrices. An intrigued reader will learn more from exploring the repository, which is open source. The chapter ends with some comments about the Application Programming Interface (API) of `SparseDiffEngine` as well as how it is used within `CVXPY`.

3.1 Overview of the engine

`SparseDiffEngine` is a small C library, roughly eighteen thousand lines of code, with a test suite of about the same size. The expression node and the routines that traverse trees of expressions are defined in `expr.c`. Each atom is implemented in its own file in the `atoms/` direc-

tory; there are forty-five of them, divided into six subdirectories based on their mathematical properties: `affine`, elementwise and bivariate functions with full or restricted domains, and a small group of `other` atoms such as the quadratic form. The matrix data structures and the numerical kernels that operate on them are in `utils/`, and `problem.c` collects the objective and constraints of a problem and implements the oracles that are passed to a solver. Atoms in the same subdirectory have Jacobians with the same structure, and their implementations share most of their code (§3.3.1). In this chapter we only discuss the affine atoms and the quadratic form, since these are the only atoms that appear in canonicalized DCP problems; the remaining classes are used by the nonlinear programming (NLP) interface of `CVXPY` [37].

Each atom implements three operations: a *forward* pass, which evaluates the atom at a point; an *init* pass, which computes the sparsity patterns of its derivatives and allocates memory for them; and a *value* pass, which computes the numerical values inside the fixed patterns. The constructor of the atom installs these operations as function pointers on the expression node. The init pass runs once per problem, while the value pass runs at every evaluation. §3.3.3 describes the benefits of this separation.

An expression node stores its dimensions, pointers to at most two children, a buffer for its value, and its derivative matrices. A simplified version of the definition in `expr.h` is shown below.

```
typedef struct expr
{
    int d1, d2, size, n_vars, refcount, var_id;
    struct expr *left;
    struct expr *right;

    double *value;   /* forward value             */
    matrix *jacobian; /* Jacobian w.r.t. the variables */

    forward_fn      forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
    /* ... Hessian oracle, workspace and caching fields ... */
} expr;
```

Trees are built bottom up, with one constructor function per atom; the Python bindings expose the same constructors. Expressions that appear multiple times can be shared, in which case the tree becomes a directed acyclic graph. Nodes are therefore reference counted, and the initialization routines return early if a node was already initialized through another

parent.

There are two leaf atoms, variables and parameters. A *variable* occupies a contiguous slice of the global variable vector, starting at its offset `var_id`; its Jacobian is a selection matrix with one unit entry per row, written once during the init pass. Unlike `CVXPY`, there is no separate constant node: constants and parameters share a single leaf type, distinguished by an identifier that is -1 for constants and, for updatable parameters, the offset of the parameter in the global parameter vector. The Jacobian of this leaf is empty, since parameters are fixed during a solve, and because parameters are leaves of the tree, the derivative structure remains valid when their values change (§1.1).

3.2 Matrix representations

The `jacobian` field of the expression node has a polymorphic type called `matrix`, with one representation for each kind of structure that we want to exploit. The representation of a Jacobian is chosen by the atom that owns it during the init pass, that is, during the symbolic phase; after that, every operation on the matrix, such as transposition, row indexing, or a chain-rule product, simply dispatches to a kernel specific to the representations involved. The default representation is a generic sparse matrix stored in the CSR format of §1.5.2 [51].

3.2.1 The permuted dense matrix

The Jacobians that appear during canonicalization are usually not sparse in the sense of having scattered, isolated nonzeros. More often they contain a dense block, typically problem data or a product involving problem data, placed at a subset of the rows and columns of a much larger matrix. As an example, consider the affine expression Ax with dense data $A \in \mathbf{R}^{50\times 50}$ in a problem with ten thousand variables. The Jacobian of this expression is the matrix A itself, stored in the fifty columns that correspond to x, with all other entries equal to zero. If this Jacobian is stored in a generic sparse format, all subsequent products go through sparse kernels, and the hardware efficiency of §1.5.1 is lost.

We therefore store such matrices in factored form. A *permuted dense* matrix is a triple (X, r, c), where $X \in \mathbf{R}^{m_0\times n_0}$ is a dense block and r and c are strictly increasing index vectors, representing

$$A = S_r^T X S_c \in \mathbf{R}^{m\times n}, \qquad A_{r_i, c_j} = X_{ij}, \tag{3.1}$$

where $S_r \in \{0,1\}^{m_0\times m}$ and $S_c \in \{0,1\}^{n_0\times n}$ are the selector matrices whose rows are the rows of the identity indexed by r and c. All other entries of A are structurally zero and never stored. Figure 3.1 shows an example with a 3×3 block inside a 6×7 matrix.

$$A = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{00} & \cdot & x_{01} & \cdot & \cdot & x_{02} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \qquad X = \begin{pmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}$$

$$r = (1, 3, 4), \qquad c = (0, 2, 5).$$

Figure 3.1 A permuted dense matrix. The dense block X is stored once, contiguously; the index vectors r and c place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored.

The block X is stored in one contiguous row-major buffer of $m_0 n_0$ doubles, and the index vectors are two integer arrays, so the amount of storage does not depend on the global dimensions. We also store the two inverse maps, which send a global index to the corresponding block index, or to -1 if there is none, so that membership can be checked in constant time. Structural operations only involve integer arithmetic on the index vectors, while all floating point operations are performed on the contiguous block.

The representation is closed under the operations that we need to perform on it. To transpose, we swap the two index vectors and transpose the block. The structural atoms, for example indexing, broadcasting, and the row partitions of `sum`, all map a permuted dense matrix to another permuted dense matrix, using integer arithmetic on the index vectors and row copies of the block. As an example, Figure 3.2 shows the result of indexing the rows $(3, 0, 4, 3)$ of the matrix in Figure 3.1. Global rows 3 and 4 correspond to block rows 1 and 2; global row 0 does not appear in the row vector, so the corresponding output row is zero; and the repeated index 3 duplicates a block row. The result is again permuted dense, with row vector $(0, 2, 3)$, the same column vector as A, and a block made of rows of X. Computing the structure takes one scan of the inverse row map, and computing the values takes three row copies.

The chain-rule product of two permuted dense matrices is also permuted dense: for permuted dense A and B, the product $B^T A$ factors as

$$B^T A = S_{c_B}^T X_B^T \underbrace{\left(S_{r_B} S_{r_A}^T\right)}_{\text{row-set intersection}} X_A S_{c_A}, \tag{3.2}$$

The middle factor is a zero-one matrix whose entry (k, l) is one exactly when $r_{B,k} = r_{A,l}$, so the sparsity analysis of the product amounts to intersecting two sorted integer arrays.

$$C = A[(3,0,4,3),:] \;=\; \begin{pmatrix} x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \\ x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \end{pmatrix} \begin{matrix} \leftarrow A_{3,:} \\ \leftarrow A_{0,:} = 0 \\ \leftarrow A_{4,:} \\ \leftarrow A_{3,:} \end{matrix}, \qquad X_C = \begin{pmatrix} X_{1,:} \\ X_{2,:} \\ X_{1,:} \end{pmatrix}$$

Figure 3.2 Row indexing of the permuted dense matrix A of Figure 3.1. One output row is zero, one block row is duplicated, and the column structure is unchanged: the result has row vector $(0, 2, 3)$, the column vector of A, and a block made of rows of X.

The result has row vector c_B and column vector c_A, which are known before any numerical values exist. For the values there are three cases. If the two row sets are equal, the product is computed with a single `dgemm` on the two blocks. If they overlap partially, the matching rows are first gathered into a workspace (allocated during init), followed by the same `dgemm`. If they are disjoint, the product is zero, and nothing is computed in the value pass. The weighted product $B^T DA$, which appears when Hessians are assembled, is handled the same way, since $S_r D S_r^T$ reduces to a small diagonal that is applied by scaling the rows of the block.

The main benefit of the factored form is that the floating point work is done by dense kernels. If the same matrix is stored in the CSR format, its products go through sparse kernels, whose indexed accesses and irregular inner loops reach only a fraction of the throughput of BLAS at the same operation count [60]. In the factored form, the value pass of each kernel consists of a few BLAS calls on contiguous buffers. The product $A^T DA$, for example, is computed with one copy, m_0 scalings, and one `dgemm`, and the cost does not depend on the global dimensions.

When an expression with matrix shape $n \times p$ is multiplied by data $A \in \mathbf{R}^{m \times n}$, the effect on the vectorized expression is that of the block diagonal operator $I_p \otimes A$. This operator is never formed. In memory, the p segments of length n of a vector x form a $p \times n$ row-major matrix $\tilde{X}$, and by a standard vectorization identity [61],

$$y = (I_p \otimes A)\, x \quad \Longleftrightarrow \quad \tilde{Y} = \tilde{X} A^T, \tag{3.3}$$

so the operator, nominally of size $pm \times pn$, is applied with a single `dgemm` and no data movement. The Jacobian products of the `left_matmul` atom use the same identity, and avoiding the explicit operator made the initialization of its sparsity pattern roughly one hundred times faster.

3.2.2 The stacked permuted dense matrix

The representation is, however, not closed under one family of products. The Jacobian of the `left_matmul` atom is $(I_p \otimes A)\, J$, where J is the Jacobian of the child expression. This matrix consists of p row blocks, each of them a permuted dense product, placed at disjoint global rows, and blocks with different column index vectors cannot be described by a single triple (X, r, c). For this reason there is a third representation. A *stacked permuted dense* matrix is a vertical union of permuted dense blocks whose row vectors are pairwise disjoint; the column vectors may overlap. The blocks share one value buffer, and the kernels of this subsection apply block by block: structural atoms modify the index vectors of each block, products distribute over the blocks with one `dgemm` each, and pairs of blocks with disjoint row sets are skipped, as in (3.2). A few operations, such as transposition, can cause row vectors to overlap; these are followed by a coalescing step that merges such blocks.

Generic sparse, permuted dense, and stacked permuted dense are the only three matrix representations in the engine.

3.3 Computational kernels

This section describes how the atoms compute their derivatives, how kernels are selected when the operands of a product have different representations, and how work is reused across evaluations.

3.3.1 Atom implementations

The six subdirectories of §3.1 group the atoms by their number of arguments and by whether their domain is all of $\mathbf{R}^n$. Atoms in the same class compute their derivatives in the same way, and the shared part of the computation is implemented once in a common file; the individual atom only supplies its forward map and its scalar derivatives. Table 3.1 summarizes the six classes. We only describe the two classes that appear in canonicalized problems.

The Jacobian of an affine atom does not depend on x. For the structural atoms, such as indexing, transposition, reshaping, stacking, and summation, the Jacobian is determined by shapes and index sets alone. Their init pass computes an index map, and their value pass gathers or copies the values of the child; for several of these atoms the value pass does nothing, because the entries were already written during init. The remaining affine atoms, such as `left_matmul`, `scalar_mult`, `convolve`, and `kron`, place problem data in the Jacobian. Their values only change when a parameter update sets the refresh flag of §3.1.

Table 3.1 The atom classes and their shared Jacobian recipes.

Class	Examples	Jacobian recipe
affine	`sum`, `index`, `left_matmul`	fixed; index maps and copies
elementwise, full domain	`exp`, `power`, `logistic`	diagonal scaling of the child Jacobian
elementwise, restricted domain	`log`, `entr`, `atanh`	diagonal at the variable's columns
bivariate, full domain	`multiply`, `matmul`	two-term chain rule
bivariate, restricted domain	`quad_over_lin`, `rel_entr`	two-term chain rule
other	`quad_form`, `prod`	bespoke

The quadratic form is the only atom outside the affine class that appears in canonicalized problems. Its gradient is affine in x, and most of its implementation belongs to the Hessian machinery, which we do not describe in this chapter.

3.3.2 Kernel dispatch

Unary operations are dispatched through the function pointer table of §3.2, based on the representation of their single operand. The chain-rule products $B^T A$ and $B^T DA$ depend on the representations of both operands, which a single table cannot express. They are instead resolved with double dispatch: a dispatch function switches on the type tags of B and then of A, and calls one of nine kernels, each consisting of an alloc and a fill function. Figure 3.3 shows the representation of the result for the nine combinations of operand types.

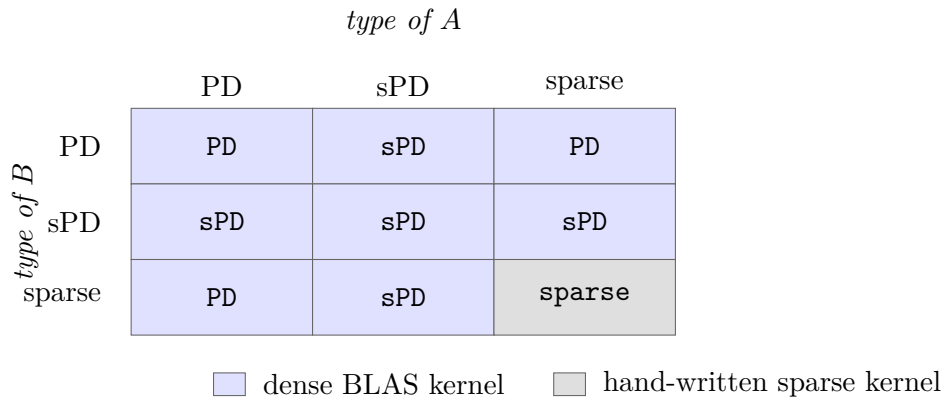


Figure 3.3 Double dispatch of the chain-rule products $B^T A$ and $B^T DA$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its color indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel.

A product with a permuted dense or stacked permuted dense operand produces a result

of the same kind, so once a dense block has been identified, it remains in a structured representation through all later products. Eight of the nine kernels are based on dense BLAS calls; in the mixed combinations, the columns of the sparse operand are gathered into the dense computation. Only the product of two generic sparse matrices uses Gustavson's algorithm [52], with the symbolic and numeric phases discussed in §1.5.2.

The number of kernels is kept small with transpose identities. No kernel transposes an operand explicitly: reading a CSR matrix through its CSC mirror gives the transpose (the mirror is built once and refreshed when needed), and a permuted dense matrix caches its transpose the first time it is requested. A plain product BA is only implemented for permuted dense B, and the block diagonal products $(I_p \otimes A)\, J$ of `left_matmul` have their own three kernels, with stacked permuted dense results. In some atoms the order of operations is chosen with the dispatcher in mind; the quadratic form, for example, computes $J^T Q J$ as $(QJ)^T J$, so that the permuted dense factor is in the operand slot that determines the kernel.

3.3.3 Reusing work across passes

The value pass never allocates memory; every buffer, including the scratch space of the kernels, is allocated during init. Beyond allocation, several other kinds of work are done only once. Sparsity patterns are computed during init: `add` merges the patterns of its children with a sorted merge, the sparse products run the symbolic phase of Gustavson's algorithm [53], and the factored products of §3.2 determine their output index vectors before any values exist. Index maps are also precomputed: transposition stores its row permutation, `kron` its per-row copy maps, and `sum` the map from child nonzeros to output entries, so the value pass performs no index arithmetic. When the index vectors of two operands do not intersect, the product is recorded as empty during init, and its value pass does nothing. Finally, values that cannot change are written once; the Jacobian of a variable, for example, is filled during init and never touched again.

Figure 3.4 illustrates this for the `sum` atom. During init, the representation of the child allocates the output and fills `idx_map`, an array that maps each nonzero of the child Jacobian to the output entry it is added to. The axis convention and all other index decisions are handled at this point. The value pass is a single loop, the same for every axis and every representation. Indexing, transposition, and `kron` work the same way, with a copy instead of an addition.

The problem level works in the same way. During init, the sparsity structures of the constraint Jacobians are concatenated into one global CSR matrix, by shifting the row pointers of each root and copying the column indices. At every evaluation, each subtree is refreshed and the

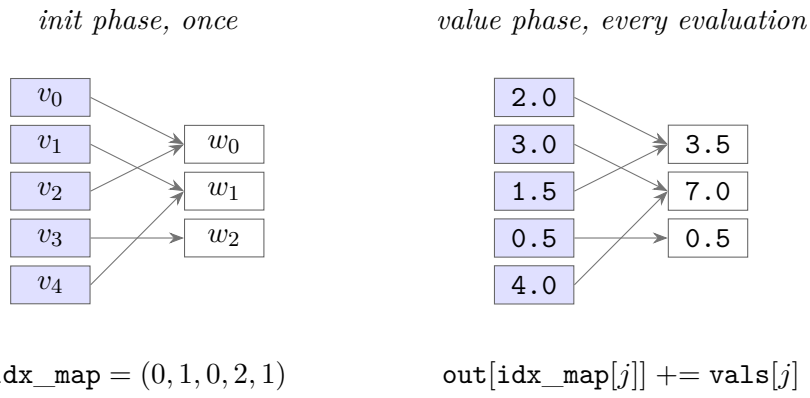


Figure 3.4 The two passes of the `sum` atom. During init, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The value passes of the other structural atoms have the same form.

value array of each root is copied into the global buffer at a fixed offset. If a root Jacobian is permuted dense, its CSR view is built once during init, and the value array of the view is a pointer to the dense block itself, so no values are ever copied or converted when the block is refilled. Affine constraints are skipped entirely after the first evaluation, since their values cannot change until the parameters do.

A parameter update therefore only involves the value pass: `update_params` writes the new values into the parameter leaves and sets the refresh flags, and the next evaluation recomputes the values through the fixed maps and kernels. In comparison, without parametric compilation [25], `CVXPY` traverses the expression graph again, recomputes every sparsity pattern, and reallocates every buffer each time a parameter changes. The difference is measured in Chapter 4.

3.4 Revisiting the limitations of matrix stuffing

§2.1 listed three implementation details that are shared by the two stuffing backends of `CVXPY`. We now describe how the engine handles each of them.

External sparse libraries. We use no external sparse library. All kernels are written by hand and specialized to structure that is known at canonicalization time, such as the selection-block Jacobians of variables and the factored products of §3.2. This made it possible to design the storage formats and the kernels around canonicalized problems instead of around generality.

Everything is sparse. Dense atom data enters the engine as a permuted dense block, remains in a structured representation through every product (Figure 3.3), and is processed by level-3 BLAS. The generic sparse type is only used for data that is actually sparse.

Quadratic objectives are separate. No dummy variables are substituted and no second stuffing pass is run. A quadratic form remains in the tree as an atom, and its contribution to P is computed by the Hessian oracle of §3.1, with the same init and value passes as everything else. A full description of the Hessian machinery is outside the scope of this chapter.

3.5 The API and the CVXPY interface

The engine is distributed as a C library with a thin Python binding called `sparsediffpy`, which exposes the node constructors. It is used in two places inside `CVXPY`. The nonlinear programming interface wraps the bindings in a class called `C_problem`, which is built from the expression trees of a problem. The canonicalization backend, which is the one measured in this thesis, is selected by passing `ignore_dpp=True`, and replaces the stuffed matrices of §2.1 with a cone program object that wraps the same `C_problem`. The code below is lightly condensed from the actual interface.

The lifecycle of a `C_problem` mirrors the two phases of this chapter:

```
prob = C_problem(problem)     # convert the tree, register parameters
prob.init_jacobian_coo()      # init phase: patterns, maps, buffers
rows, cols = prob.get_jacobian_sparsity_coo() # structure, fetched once

prob.objective_forward(x)     # value phase, repeated per iterate
prob.constraint_forward(x)
grad = prob.gradient()
vals = prob.eval_jacobian_vals() # values aligned with (rows, cols)
```

The sparsity pattern is fetched once and does not change afterwards; the later calls return value arrays that are aligned with it.

The canonicalization backend extracts the problem data, an objective $\frac{1}{2}x^TPx + q^Tx + d$ and constraints $Ax + b$ that must lie in a cone, by evaluating the oracles at $x = 0$. The forward passes give d and b, the gradient gives q, and the Jacobian and Hessian values fill the fixed patterns of A and P. Parameter updates go through a single method:

```
def apply_parameters(self, id_to_value):
    theta = flatten_and_concatenate(id_to_value)
    if theta == self.cached_theta:    # same values: reuse everything
```

```
        return self.P, self.q, self.d, self.A, self.b
    self.c_problem.update_params(theta) # numeric phase only
    self.P, self.q, self.d, self.A, self.b = self.extract() # at x = 0
    self.cached_theta = theta
    return self.P, self.q, self.d, self.A, self.b
```

Repeated solves with unchanged parameter values return the cached matrices; an actual update costs one run of the value pass.

The caching follows the same pattern as in §3.3.3. The `C_problem` and its sparsity structures live as long as the problem family, the extracted matrices are kept until the parameter values change, and only numerical values are recomputed. After construction, nothing in the Python layer traverses an expression tree. Chapter 4 compares the engine with the systems reviewed in Chapter 2.

References. The row-wise sparse product and the symbolic and numeric split are due to Gustavson [52]; see also [53]. The books of Davis [51] and Saad [50] cover the compressed formats and their kernels. The BLAS interface was developed in [57,58], and [60] explains the hardware behavior behind its performance. Griewank and Walther [46] treat the computation of Jacobians on expression graphs. The idea of compiling a parametric problem family once also appears in DPP [25], where a cached parameter map plays the role of the init pass.

CHAPTER 4 Benchmark results

This chapter presents benchmark results which compare the `diffengine` backend (the `CVXPY` implementation based on `SparseDiffEngine`) with the systems reviewed in Chapter 2, keeping its organization: the `cvxcore` matrix-stuffing baseline of §2.1, the two structure-discovery systems of §2.2 (`CasADi` and the `SCT` sparse-AD stack), and the two declared-structure modeling systems of §2.3 (`AMPL` and `JuMP`). All comparisons run on the 25 problems of `CVXPY`'s public benchmark suite (`https://github.com/cvxpy/benchmarks`). These convex optimization problems cover many fields, ranging from portfolio models, robust regression, and semidefinite programs to simple LPs and QPs that are close to standard form. The rest of the chapter first presents some technical details about the benchmark setup, followed by the comparison results which are paired with commentary and observations.

4.1 Benchmark Setup

Every benchmark measures the construction of the problem data (A, b, c, P); no solver is invoked. For the two `CVXPY` backends this corresponds to a call to `get_problem_data` with the target solver fixed to `Clarabel` [23]: the `CPP` (`cvxcore`) baseline runs on the default DPP path, while `diffengine` is selected by passing `ignore_dpp=True`. For `CasADi` and the `SCT` stack, we time the extraction of the same (A, b, c, P) by differentiation of an equivalent expression graph, following the conventions described in Section 4.4. For `JuMP` and `AMPL`, we time the assembly of the same data from an equivalent declared-structure model, following the conventions described in Section 4.5.

Every comparison reports up to two timings: a *single* solve, where the problem data are constructed once from scratch, and, on the subset of parametric problems, a *re-solve*, where the problem data are reconstructed after reassigning random parameter values on a cached problem.

All measurements run single-threaded (BLAS thread pools pinned to one thread) on a 2018 MacBook Pro with a 6-core Intel Core i7-8850H at 2.6 GHz and 16 GB of RAM. The software versions are:

- `Python` 3.14,
- `CVXPY` 1.10 (development version),
- `sparsediffpy` 0.6.1,

- `NumPy` 2.4 [39] and `SciPy` 1.17 [54],
- `CasADi` 3.7.2 [12],
- the `SCT` sparse-AD stack [72, 78] on `Julia` 1.10.5 (single-threaded): `SparseConnectivityTracer` 1.2.2, `SparseMatrixColorings` 0.4.27, `DifferentiationInterface` 0.7.20, and `ForwardDiff` 1.4.1,
- `JuMP` 1.30.1 on `MathOptInterface` 1.51.2 [11, 30],
- `amplpy` with the AMPL Community Edition engine (2026), read back through an ASL interface built from source (`Pyomo` 6.10.1 / `PyNumero`) [29, 80].

Each reported time is the median of three new compilations; the re-solve columns are medians of twenty re-evaluations. Before any timing, every tool is verified to reproduce `CVXPY`'s problem data bit-identically ($\max|\Delta| = 0$) at full benchmark size. The `cvxcore` and `CasADi` comparisons cover all 25 problems; the `SCT` harness ports four representative problems; the `JuMP` and `AMPL` comparisons cover the 21-problem subset that has been ported to a tool-neutral lowered form (the three `kron`-based problems and CVaR are not ported). The scripts which run the benchmarks, the models for each tool, and all recorded timings are available at `https://github.com/SparseDifferentiation/Jacobian-accumulation` under the `canonicalization/` directory.

4.2 Single-solve compilation against `cvxcore`

A fresh problem is formed and canonicalized once from scratch (Table 4.1).

`diffengine` is faster on 24 of the 25 problems, with speedups between 1.2× and 124× and a geometric mean of 4.2×. The largest speedups occur where `cvxcore`'s per-expression tensor construction dominates the compilation time, as in the portfolio and regression models with many scalar constraints. UnconstrainedQP, a tiny problem wrapped in a large `kron` expression, is the extreme case at 124×. The speedups shrink on problems whose cost is a single large data copy, such as OptimalAdvertising and CVaR. Finally, the compilation of ConvexPlasticity, is dominated by `CVXPY`'s Python-side reductions rather than by either backend.

4.3 Parametric compilation against `cvxcore`

On the subset of parametric problems, we report both the first solve and the subsequent re-solve time after reassigning random parameter values (Table 4.2).

Table 4.1 Cold compilation time across all 25 benchmarks, `diffengine` versus `cvxcore`. The ratio is `diffengine`/`CPP`; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

Benchmark	`CPP` (s)	`diffengine` (s)	Ratio
UnconstrainedQP	9.32	0.08	**0.01**
Cajas	19.66	1.06	0.05
FactorCovarianceModel	5.24	0.73	0.14
SVMWithL1Regularization	11.16	1.60	0.14
Yitzhaki	3.07	0.47	0.15
HuberRegression	10.97	1.61	0.15
LeastSquares	8.87	1.42	0.16
SimpleQP	10.49	1.69	0.16
SimpleScalarParametrizedLP	5.29	0.92	0.17
SimpleLP	30.07	5.29	0.18
SDPSegfault1132	59.75	12.47	0.21
SimpleFullyParametrizedLP	2.12	0.45	0.21
ParametrizedQP	10.19	2.25	0.22
TvInpainting	3.57	1.06	0.30
SlowPruning	6.88	2.15	0.31
SemidefiniteProgramming	5.38	1.93	0.36
ConeMatrixStuffing	10.34	3.76	0.36
QuantumHilbertMatrix	3.89	1.64	0.42
SmallMatrixStuffing	6.52	2.77	0.42
ParamSmallMatrixStuffing	0.48	0.21	0.43
ParamConeMatrixStuffing	0.31	0.14	0.45
Murray	5.79	3.43	0.59
CVaR	34.80	25.31	0.73
OptimalAdvertising	6.06	5.20	0.86
ConvexPlasticity	2.09	2.13	**1.02**
Geometric mean			**0.24**

Table 4.2 Parametric compilation, `diffengine` versus `cvxcore`: first solve and warm re-solve. The ratio is `diffengine`/`CPP` on the re-solve; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

		First solve (s)		Re-solve (s)		
Benchmark	#par	`CPP`	`diffengine`	`CPP`	`diffengine`	Ratio
ParamConeMatrixStuffing	3	0.31	0.14	0.0030	0.0010	**0.33**
ParamSmallMatrixStuffing	3	0.48	0.21	0.0048	0.0017	0.35
SimpleFullyParametrizedLP	1	2.12	0.45	0.230	0.123	0.54
ParametrizedQP	2	10.19	2.25	1.361	0.959	0.70
SimpleScalarParametrizedLP	1	5.29	0.92	0.330	0.251	0.76
ConvexPlasticity	3	2.09	2.13	0.668	0.666	1.00
FactorCovarianceModel	2	5.24	0.73	0.144	0.277	1.92
SVMWithL1Regularization	1	11.16	1.60	0.263	0.530	**2.01**
Geometric mean						**0.78**

`diffengine` is faster on the re-solve for six of the eight parametric problems, and on the first solve for all but ConvexPlasticity. Its re-solves are fast because the compiled program is cached, so only the parameter-dependent coefficient entries are re-evaluated. The DPP baseline instead multiplies its cached parameter tensor by the new parameter vector, which is hard to beat when that tensor is small and dense. This explains the two rows where DPP remains around 2× faster: a scalar parameter whose effect is spread across many coefficients (SVM's regularization weight, FactorCovariance's risk aversion) is the ideal case for the tensor product.

4.4 Comparison against structure discovery: `CasADi` and `SCT`

`CasADi` [12] extracts problem data by algorithmic differentiation of a symbolic expression graph. Since `CasADi` has no notion of disciplined convex programming, it cannot canonicalize the high-level models; for a fair comparison we hand it the *already-lowered* formulation: the same epigraph variables and the same conic rows that `CVXPY`'s canonicalization frontend produces, written as `CasADi` `MX` expressions $f(z,\theta)$ (objective) and $g(z,\theta) = b - Az$ (constraint slacks), where z stacks the original and epigraph variables and θ collects the parameters.

The problem data are recovered exactly the way `CasADi`'s own quadratic-programming interface `qpsol` does [82], by differentiating the graph without invoking any solver:

$$c = \nabla_z f \Big|_{z=0}, \qquad P = \frac{\partial(\nabla_z f)}{\partial z}, \qquad A = -\frac{\partial g}{\partial z}, \qquad b = g \Big|_{z=0}.$$

These four quantities are wrapped in a `casadi.Function` which maps θ to (P, c, A, b). Computing P as the Jacobian of the gradient is `CasADi`'s fastest path on these problems.

For `CasADi`, the single-solve timing covers differentiation, function construction, and the first numeric evaluation on a freshly built graph, while a re-solve is one further evaluation of the compiled function at the new parameter values. Graph construction itself is excluded, mirroring the `diffengine` timing, which also starts from a built expression tree. We verified on every benchmark that the two systems produce identical (P, c, A, b) at full benchmark size, so the tables below time the construction of the same matrices. All 25 problems are compared, with CVaR reported as a did-not-finish.

The same lowered formulations also drive the generic Julia sparse-AD stack of §2.2.1: the sparsity patterns of A and P are traced by `SparseConnectivityTracer`, compressed by `SparseMatrixColorings` (unidirectional distance-2 coloring for A, star coloring for P), and the values are then recovered with compressed `ForwardDiff` sweeps driven by `DifferentiationInterface`. The harness ports four representative problems: SimpleQP, LeastSquares, and ParametrizedQP at their benchmark sizes, plus CVaR at a reduced slice of 8192 scenarios (the full 131 072-scenario instance exceeds the machine's memory through every Julia path). Julia's JIT compilation is warmed up on a small instance and excluded, and each problem reports the faster of the two Hessian routes (native sparse Hessian versus Jacobian of the gradient), mirroring the `CasADi` rule.

These benchmarks are not the most favorable showcase for `CasADi`. It is built to differentiate arbitrary programs, whose derivatives genuinely change with the evaluation point, and the canonicalization of affine and quadratic problems, where every derivative is constant, is a degenerate special case of that task. `CasADi` is also designed for workflows which construct a function once and then evaluate it many times, so the one-time extraction cost measured here is amortized away in its intended use.

`diffengine` is fastest where the lowered matrices contain large dense blocks: it knows every atom's Jacobian block and its sparsity pattern in closed form, while `CasADi` must discover the pattern and reconstruct the values through seeded derivative sweeps, whose number a dense block forces to the full dimension. CVaR is the extreme case: its constraint matrix contains a dense $131\,072 \times 768$ scenario block, which forces roughly 770 derivative sweeps during the Jacobian construction, and the extraction did not finish within 10 h.

The `SCT` column shows that this cost belongs to the paradigm, not to `CasADi`'s implementation of it: the state-of-the-art generic stack is slower still on all three shared problems, and its stage instrumentation places the bulk of the time in the greedy coloring itself (52 s of the 85.5 s on SimpleQP, 86 s of the 111.5 s on ParametrizedQP). A dense $m \times n$ block makes

Table 4.3 Single problem-data extraction, `diffengine` versus the two structure-discovery systems: the `SCT` stack on its four ported problems and `CasADi` on all 25 benchmarks. Dashes in the `SCT` column mark problems not ported to the Julia harness. The ratio is `diffengine`/`CasADi`; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

Benchmark	`SCT` (s)	`CasADi` (s)	`diffengine` (s)	Ratio
ParametrizedQP	111.5	771.5	2.52	**0.003**
FactorCovarianceModel	—	250.4	2.05	0.008
SVMWithL1Regularization	—	145.5	1.79	0.012
SemidefiniteProgramming	—	66.4	2.07	0.031
SimpleQP	85.5	53.7	1.70	0.032
LeastSquares	90.5	36.6	1.62	0.044
HuberRegression	—	42.4	1.88	0.044
Yitzhaki	—	8.72	0.47	0.054
SlowPruning	—	38.8	2.80	0.072
SimpleLP	—	38.1	5.02	0.132
TvInpainting	—	8.49	1.14	0.134
Murray	—	25.9	3.52	0.136
SimpleFullyParametrizedLP	—	2.36	0.48	0.204
SimpleScalarParametrizedLP	—	4.57	0.98	0.214
Cajas	—	4.88	1.12	0.229
ConeMatrixStuffing	—	6.10	3.94	0.646
OptimalAdvertising	—	9.47	6.16	0.650
SDPSegfault1132	—	17.65	14.61	0.83
SmallMatrixStuffing	—	2.77	2.75	0.99
QuantumHilbertMatrix	—	1.41	1.57	1.11
UnconstrainedQP	—	0.021	0.074	3.49
ConvexPlasticity	—	0.13	2.29	17.3
ParamSmallMatrixStuffing	—	0.007	0.21	30.8
ParamConeMatrixStuffing	—	0.004	0.14	**39.7**
CVaR	—	DNF	25.3	—
Geometric mean				**0.24**

every pair of columns collide, so its column-intersection graph is a clique: no coloring can compress it and the sweep count equals n at any implementation quality. Even the reduced CVaR slice costs the stack 51.3 s, because the epigraph-sum row makes all 8 193 columns of the scenario block pairwise conflict.

On the subset of parametric problems, we report both the first extraction and the subsequent re-solve time after reassigning random parameter values (Table 4.4), mirroring Table 4.2.

Table 4.4 Parametric extraction, `diffengine` versus `CasADi`: first extraction and warm re-solve. The ratio is `diffengine`/`CasADi` on the re-solve; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

		First (s)		Re-solve (s)		
Benchmark	#par	`CasADi`	`diffengine`	`CasADi`	`diffengine`	Ratio
FactorCovarianceModel	2	250.4	2.05	3.956	0.567	**0.14**
SVMWithL1Regularization	1	145.5	1.79	0.548	0.491	0.90
ParametrizedQP	2	771.5	2.52	0.938	0.942	1.00
SimpleScalarParametrizedLP	1	4.57	0.98	0.124	0.236	1.90
SimpleFullyParametrizedLP	1	2.36	0.48	0.062	0.120	1.95
ParamSmallMatrixStuffing	3	0.007	0.21	0.0007	0.0020	3.0
ParamConeMatrixStuffing	3	0.004	0.14	0.0003	0.0011	3.5
ConvexPlasticity	3	0.13	2.29	0.0067	0.707	**105.5**
Geometric mean						**2.19**

The re-solve plays to `CasADi`'s strengths. A re-solve only runs the compiled function: the constant values are baked in at construction time, so only the entries which actually depend on θ are recomputed, with no Python in the loop. This is about 2× faster in geometric mean than the `diffengine` re-solve, which still traverses `CVXPY`'s solve path, and dramatically faster when the parameters only change b (ConvexPlasticity). `diffengine` remains faster when a parameter reshapes the quadratic itself (FactorCovarianceModel), where `CasADi` must re-gather a large parameter-dependent P on every evaluation. The `SCT` stack sits at the opposite end: it compiles no parametric function at all, so a warm re-solve re-runs its compressed sweeps with cached pattern and coloring, costing 17.9 s on ParametrizedQP where both `CasADi` and `diffengine` re-solve in under a second.

4.5 Comparison against declared structure: `AMPL` and `JuMP`

The two modeling systems of §2.3 never discover structure: their input contract is the explicit coefficient blocks themselves, assembled at model-build time from the user's data arrays.

They therefore do strictly less work than either backend of `CVXPY`, which starts from a DCP atom tree, and less than the AD tools, which must recover the coefficients from an expression graph. The comparison is still informative in both directions: it establishes the floor that pure data-structure assembly sets for problem-data construction, and it prices the interface through which each system reaches that floor. As everywhere in this chapter, model construction itself is excluded from the headline and reported alongside the raw results: for `JuMP` this excluded macro build is where the coefficient gathering actually happens (7–9 s on typical problems, 29 minutes for SimpleLP at $n = 10^7$), and for `AMPL` it is the transfer of the data arrays into the AMPL engine.

The `JuMP` measurement is one `MOI.copy_to` call into a `MatrixOfConstraints` cache — the assembly step that conic solver wrappers such as `Clarabel.jl` execute inside `attach_optimizer`; no differentiation occurs anywhere on this path. The `AMPL` measurement covers its full pipeline to the same numeric data: the C translator emits a binary `.nl` file (with presolve disabled so the instance stays verbatim), the ASL parses it back and builds its evaluation tapes, and the derivative system produces the constraint values, Jacobian, Hessian of the Lagrangian, and objective gradient. The file round trip is included in the headline because the `.nl` file *is* ASL's interface — no in-memory API exists — and Table 4.5 splits it into its write, read, and derivative-evaluation stages. Both systems reproduce `CVXPY`'s data bit-identically before timing. Coverage: `JuMP` completes 20 of the 21 ported problems and `AMPL` 19 (SlowPruning's 8.4-million-nonzero `kron` block exceeds this machine's memory through both modeling layers, and `AMPL` *rejects* the OptimalAdvertising instance, whose generator produces a non-finite coefficient at benchmark size — it is the only compared system that refuses the poisoned data rather than propagating `NaN`).

Two observations structure the table. First, `JuMP`'s `copy_to` is the assembly floor: faster than `diffengine` on 17 of the 20 problems it completes, and sub-second on every problem except SimpleLP at $n = 10^7$. The gap to `diffengine` is nevertheless a few seconds at most in absolute terms, and it buys very different amounts of work: `copy_to` serializes coefficients that the excluded macro build already gathered, while `diffengine` performs the full canonicalization of Chapter 3 from the DCP atom tree within a small factor of that floor.

Second, the `AMPL` split makes the cost of the file interface explicit: the `.nl` write and read together account for 95–99 % of the pipeline on every problem of consequence, at roughly one second per 10–20 MB in each direction, while the *derivative* work — the stage that corresponds to what the structure-discovery systems pay tens or hundreds of seconds for — stays below 2 s everywhere except the 10^7-variable SimpleLP (9.7 s, linear in the problem size). This is the quantitative form of the positioning in §2.3.1: declared structure makes derivative

Table 4.5 Single problem-data extraction against the declared-structure systems, on the 21-problem ported subset. The `AMPL` pipeline is split into its stages: binary `.nl` write by the C translator, ASL read (parse and tape build), and derivative evaluations (constraints, Jacobian, Hessian of the Lagrangian, gradient). Stage columns come from one instrumented run while totals are medians of three, so stages need not sum exactly to the total; on the millisecond-scale problems the instrumented cold-cache stages exceed the warmer median total. Dashes: SlowPruning exceeds memory in both systems; `AMPL` rejects OptimalAdvertising (non-finite coefficient).

	`AMPL` (s)					
Benchmark	write	read	deriv.	total	`JuMP` (s)	`diffengine` (s)
SimpleLP	58.7	49.1	9.72	120.6	42.0	5.02
SVMWithL1Regularization	25.8	24.0	1.06	50.0	0.59	1.79
SimpleQP	16.2	16.5	1.98	35.6	0.76	1.70
FactorCovarianceModel	11.6	13.3	0.72	26.1	0.37	2.05
SimpleScalarParametrizedLP	11.1	8.94	0.32	21.1	1.16	0.98
LeastSquares	8.71	8.32	0.42	18.9	0.79	1.62
ParametrizedQP	9.34	8.76	0.53	18.6	0.79	2.52
Yitzhaki	3.48	7.44	0.37	18.3	0.16	0.47
HuberRegression	8.55	8.09	0.48	17.5	0.71	1.88
TvInpainting	8.21	6.42	0.22	14.9	0.85	1.14
Cajas	6.54	6.99	0.25	13.8	0.11	1.12
Murray	5.97	5.54	0.20	11.7	0.17	3.52
SimpleFullyParametrizedLP	4.48	4.48	0.17	9.08	0.53	0.48
SemidefiniteProgramming	1.53	1.48	0.11	3.13	0.12	2.07
ConvexPlasticity	0.20	0.22	0.02	0.41	0.017	2.29
SmallMatrixStuffing	0.011	0.027	0.001	0.039	0.001	2.75
ConeMatrixStuffing	0.044	0.096	0.003	0.031	0.002	3.94
ParamSmallMatrixStuffing	0.006	0.054	0.001	0.013	0.0002	0.21
ParamConeMatrixStuffing	0.005	0.054	0.001	0.012	0.0002	0.14
OptimalAdvertising	—	—	—	—	0.16	6.16
SlowPruning	—	—	—	—	—	2.80

extraction essentially free, and what `AMPL` pays instead is its 1990 process-decoupling design, a price that is invisible when one instance is solved once and decisive when canonicalization repeats. The same file interface is also why `AMPL` scales: its translator streams, where `Pyomo`'s object-graph modeling layer cannot instantiate SimpleLP at $n = 10^7$ on this machine at all. Overall, `diffengine` is faster than the full `AMPL` pipeline on 14 of the 19 completed problems (geometric-mean ratio 0.37), while doing the canonicalization work that `AMPL` assumes already done.

CHAPTER 5 CONCLUSION

5.1 Summary of Works

5.2 Limitations

5.3 Future Research

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[2] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[3] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[4] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[5] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[6] J. Bisschop, *AIMMS optimization modeling*, 2006.

[7] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[8] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[9] M. L. Bynum *et al.*, *Pyomo—Optimization Modeling in Python*. Springer, 2021.

[10] W. E. Hart *et al.*, "Pyomo: modeling and solving mathematical programs in python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[13] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[14] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[15] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[16] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[17] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[18] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[19] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[20] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[21] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[22] D. Ge, Q. Huangfu, Z. Wang, J. Wu, and Y. Ye, "Cardinal Optimizer (COPT) user guide," *arXiv preprint arXiv:2208.14314*, 2022.

[23] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.

[24] A. Ben-Tal and A. Nemirovski, *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*. SIAM, 2001.

[25] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[26] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *IMA Journal of Applied Mathematics*, vol. 13, no. 1, pp. 117–119, 1974. [Online]. Available: https://doi.org/10.1093/imamat/13.1.117

[27] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[28] D. M. Gay, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[29] ——, "Hooking your solver to AMPL," Bell Laboratories, Tech. Rep. 97-4-06, 1997, reference for the AMPL Solver Library (ASL).

[30] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[31] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

[32] E. M. B. Smith, "On the optimal design of continuous processes," Ph.D. dissertation, Imperial College London (University of London), London, 1996, ph.D. dissertation.

[33] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[34] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[35] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[36] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[37] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," 2025.

[38] W. Zhang, P. Nobel, A. Jeendgar, R. Murray, P. Schiele, and S. Diamond, "CVXPY 1.9: Recent advances in optimization modeling software," https://www.cvxpy.org/_static/cvxpy-1.9.pdf, 2026, preprint.

[39] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[40] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[41] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[42] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[43] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[44] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario. [Online]. Available: https://www.maplesoft.com

[45] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[46] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[47] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[48] U. Naumann, "Optimal Jacobian accumulation is NP-complete," *Mathematical Programming*, vol. 112, no. 2, pp. 427–441, 2008. [Online]. Available: https://doi.org/10.1007/s10107-006-0042-z

[49] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[50] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[51] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[52] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software*, vol. 4, no. 3, pp. 250–269, 1978.

[53] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Advances in Computational Mathematics*, vol. 1, no. 1, pp. 127–137, 1993.

[54] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[55] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[56] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.

[57] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for Fortran usage," *ACM Transactions on Mathematical Software*, vol. 5, no. 3, pp. 308–323, 1979.

[58] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Transactions on Mathematical Software*, vol. 16, no. 1, pp. 1–17, 1990.

[59] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*, 3rd ed. Philadelphia, PA: SIAM, 1999.

[60] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[61] J. R. Magnus and H. Neudecker, *Matrix Differential Calculus with Applications in Statistics and Econometrics*, 3rd ed., ser. Wiley Series in Probability and Statistics. John Wiley & Sons, 2019.

[62] P. S. Dwyer and M. S. Macphail, "Symbolic matrix derivatives," *The Annals of Mathematical Statistics*, vol. 19, no. 4, pp. 517–534, 1948.

[63] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[64] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[65] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," 2010. [Online]. Available: http://eigen.tuxfamily.org

[66] M. ApS, "MOSEK modeling cookbook," 2026.

[67] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[68] A. Griewank, D. Juedes, and J. Utke, "Algorithm 755: ADOL-C: a package for the automatic differentiation of algorithms written in C/C++," *ACM Transactions on Mathematical Software*, vol. 22, no. 2, pp. 131–167, 1996. [Online]. Available: https://doi.org/10.1145/229473.229474

[69] B. M. Bell, "CppAD: A package for C++ algorithmic differentiation," 2023, computational Infrastructure for Operations Research. [Online]. Available: https://github.com/coin-or/CppAD

[70] T. F. Coleman and J. J. Moré, "Estimation of sparse Jacobian matrices and graph coloring problems," *SIAM Journal on Numerical Analysis*, vol. 20, no. 1, pp. 187–209, 1983. [Online]. Available: https://doi.org/10.1137/0720013

[71] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J. on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[72] A. Hill and G. Dalle, "Sparser, better, faster, stronger: Sparsity detection for efficient automatic differentiation," *arXiv preprint arXiv:2501.17737*, 2025. [Online]. Available: https://arxiv.org/abs/2501.17737

[73] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[74] C. F. Kwok, D. Zhu, and L. Jacobi, "An analysis of vectorised automatic differentiation for statistical applications," *Stats*, vol. 8, no. 2, 2025. [Online]. Available: https://doi.org/10.3390/stats8020040

[75] Y. Zou and F. Jiang, "Vectorized sparse second-order forward automatic differentiation for optimal control direct methods," *arXiv preprint arXiv:2506.11537*, 2025. [Online]. Available: https://arxiv.org/abs/2506.11537

[76] M. Chi and S. Wen, "torch-sla: Differentiable sparse linear algebra with adjoint solvers and sparse tensor parallelism for PyTorch," *arXiv preprint arXiv:2601.13994*, 2026. [Online]. Available: https://arxiv.org/abs/2601.13994

[77] A. Shaikhha, M. Huot, and S. Hashemian, "∇sd: Differentiable programming for sparse tensors," *arXiv preprint arXiv:2303.07030*, 2023. [Online]. Available: https://arxiv.org/abs/2303.07030

[78] A. Hill and G. Dalle, "SparseConnectivityTracer.jl: Jacobian and Hessian sparsity detection via operator overloading," https://github.com/adrhill/SparseConnectivityTracer.jl, 2025, julia package, companion software to the sparsity detection paper.

[79] ——, "An illustrated guide to automatic sparse differentiation," ICLR 2025 Blogposts Track, 2025. [Online]. Available: https://iclr-blogposts.github.io/2025/blog/sparse-autodiff/

[80] AMPL Optimization Inc., "amplpy: Python API for AMPL," https://amplpy.ampl.com, 2025, python interface to the AMPL modeling language.

[81] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[82] CasADi developers, "CasADi user guide," https://web.casadi.org/docs/, 2025.